Enterprise AI Platform — Architecture & Engineering Handbook

Author's Edition — June 2026

> *This handbook is written for the engineer who looks at an AI demo and immediately asks: What happens when it fails? How do I extend it? Why was it designed this way? If you're that engineer, start here.*

Contents

---

1. Why This Platform Exists

Most AI projects begin the same way. Someone writes a Python script that calls an LLM API. They demo it on a laptop with one document. It works. Everyone is excited.

Then the engineering questions start.

What happens when Qdrant is down? Which prompt version generated that answer? How do I add Anthropic alongside Ollama without rewriting retrieval? Can I trace why this answer cited that document?

These questions expose the gap between an AI demo and an AI platform. A demo needs one happy path. A platform needs every unhappy path handled deliberately.

This project closes that gap. It is a working demonstration of how a team of experienced software engineers integrates AI into enterprise infrastructure — not as a feature bolted onto an existing system, but as infrastructure designed from first principles.

The result is a modular monolith: 9 Maven modules, a single deployable, compile-time boundaries that enforce dependency direction. It is not a SaaS product. It is not a chatbot framework. It is a foundation — the kind of codebase another engineering team could adopt as the starting point for their own AI-powered application.

What's in scope: multi-provider AI orchestration, hybrid search across keyword, vector, and graph sources, automatic semantic enrichment during ingestion, GraphRAG with Neo4j traversal, versioned prompt management, queryable model capabilities, automated evaluation of every answer, full explainability metadata, a reusable workflow engine, production metrics, and 157 automated tests.

What's deliberately absent: microservices, multi-agent systems, model fine-tuning, user management, billing. These are valid concerns. They would also distract from the core architecture — the part this project exists to demonstrate.

> ***Design Rationale:*** *Modular Monolith over Microservices — see ADR-001. Compile-time boundaries provide the architectural benefits of modularity without distributed systems complexity. Modules can be extracted into services if needed later. The reverse — combining microservices into a monolith — is far harder.*

---

2. The Principles

A platform of this size makes hundreds of small decisions. Eight principles keep them coherent. When a proposal conflicts with a principle, the proposal loses — or the principle is amended explicitly, never silently.

AI Is Infrastructure

LLMs, embedding models, vector databases, knowledge graphs — treat them the way you treat PostgreSQL. Abstract them behind interfaces. Select them via configuration. Swap them without touching business logic.

A `DocumentService` never calls `Ollama`. It calls `EnrichmentService`. Whether enrichment runs on Ollama, OpenAI, or a regex fallback is not a business concern. It is an infrastructure concern. This pattern repeats everywhere.

Graceful Degradation, Not Hard Failures

In production, external services fail. Designing for this from day one means the platform is useful even when individual components disappear. Not as an afterthought — as a first-order design constraint.

Every external dependency is optional. The platform starts with only PostgreSQL. Add Qdrant: vector search activates. Add Neo4j: the graph populates. Remove any: the platform continues — reduced capabilities, logged clearly, never silently. Conditional beans, `ObjectProvider<T>` injection, `NoOp*` fallbacks, and `isAvailable()` checks are built into every provider boundary.

> ***Operational Note:*** *Health indicators for every external service are at `/actuator/health`. If a service is down, the endpoint shows it, metrics record it, logs explain why — and the platform keeps running.*

Explainability by Default

If this platform generates an answer, it can explain itself. Which intent was classified. Which retrieval strategy was selected. Which prompt template — and which version — was used. Which provider and model ran inference. How many chunks and graph nodes were retrieved. What evaluation scores resulted.

This is not a feature flag. It is built into the orchestration layer and runs during normal execution. There is no separate "explain this" pass. Every code path produces metadata.

Documents Are the Knowledge Source

Users upload documents. The platform builds the knowledge. There is no manual knowledge base, no CRUD interface for entities. The enrichment engine extracts entities, concepts, and relationships automatically during ingestion. The knowledge graph is auto-generated.

An earlier version of this project included a manual knowledge base. It was removed. The document corpus is the knowledge source. Everything else is derived.

Domain Independence

The core contains no assumptions about legal documents, financial reports, or any industry. Domain customization lives behind the `DomainConfiguration` SPI. The included Document Intelligence application provides defaults. New domains add their own configuration — no core code changes needed.

Testing as Architecture Documentation

Tests describe behavior, not implementation. A Staff Engineer should understand the platform by reading the test suite. Test names read like specifications: "routes openai:gpt-4o to openai provider" — not "testProviderRouter."

Production Readiness

Observability, health checks, and configuration are not afterthoughts bolted on before release. They are first-class concerns designed alongside the features they observe.

Modular Monolith

Module boundaries enforced at compile time. Dependency direction flows one way. Higher modules depend on lower modules — never the reverse.

> *All eight principles have dedicated ADRs in Appendix A.*

---

3. How the Platform Fits Together

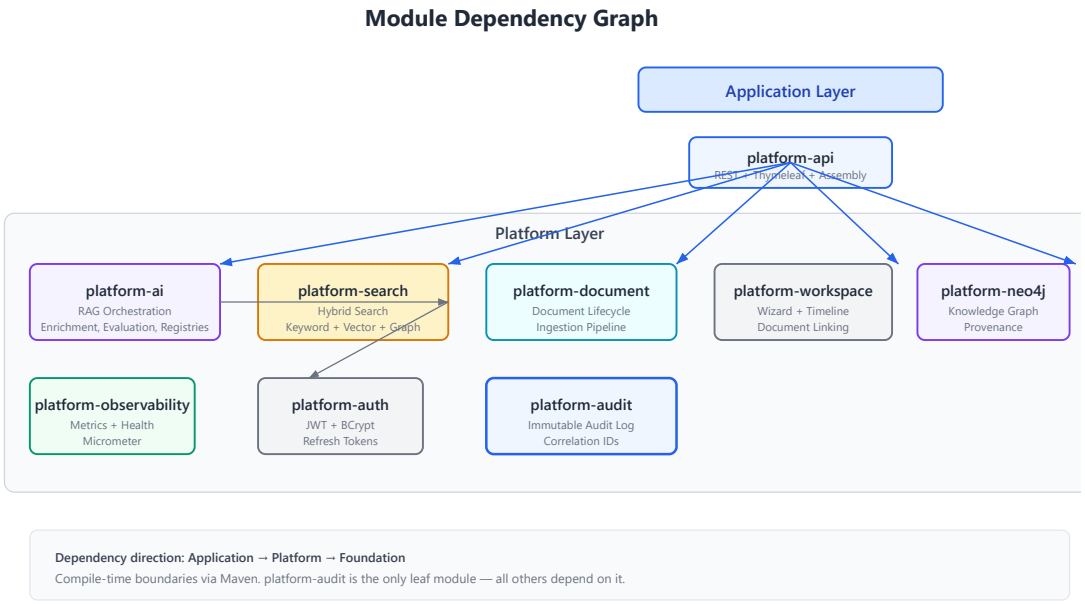

Figure 3.1. Nine modules in three layers. Application depends on Platform. Platform depends on Foundation. Arrows point in the direction of dependency — `platform-api` depends on everything; nobody depends on `platform-api`.

Three layers, nine modules, one direction of dependency.

At the bottom, `platform-audit` is the only leaf module. Every other module depends on it. It provides immutable audit logging with correlation IDs that thread through every request — from HTTP arrival to AI answer. Nothing above it needs to know how auditing works. They call the interface. The implementation handles the rest.

The middle layer holds seven platform modules. Each has exactly one responsibility: authentication, document lifecycle, hybrid search, AI orchestration, Neo4j graph persistence, workspace management, observability. When a module grows large — as the AI module has — its internal package structure provides separation without adding Maven modules. The goal is clarity, not a specific module count.

At the top, `platform-api` is the assembly point. REST controllers, Thymeleaf templates, DTOs, and the configuration class that conditionally activates provider beans based on which infrastructure is available. Future applications — contract intelligence, financial analysis, compliance — would create their own assembly modules on the same platform foundation.

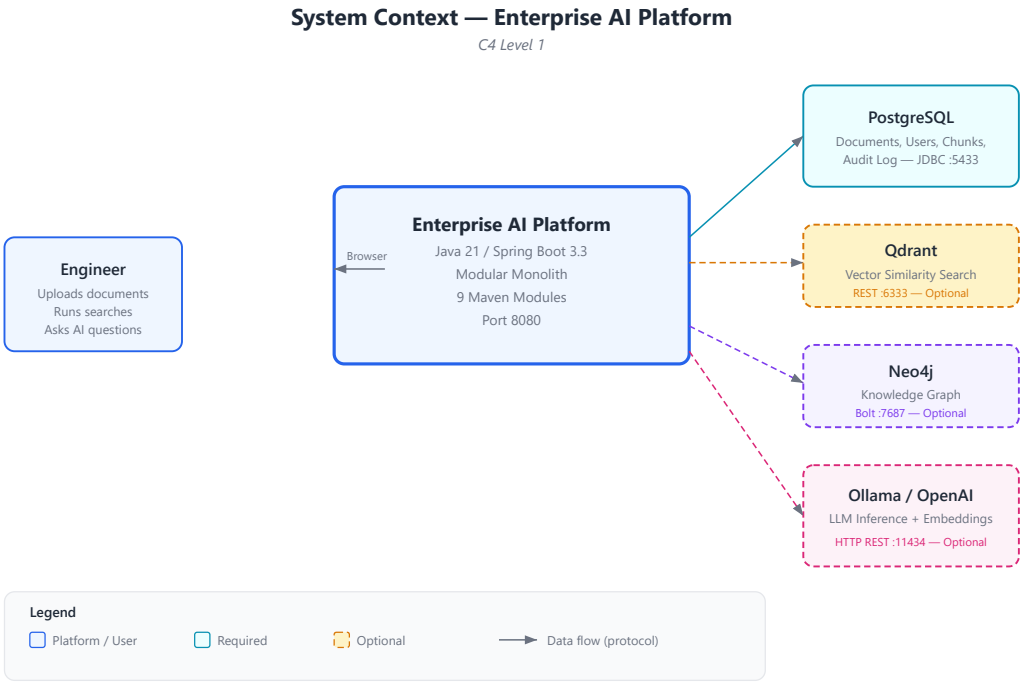

Figure 3.2. The platform communicates with four external systems. PostgreSQL is the only hard requirement. Everything else — Qdrant, Neo4j, Ollama/OpenAI — is optional. This is not a compromise. It is a design goal.

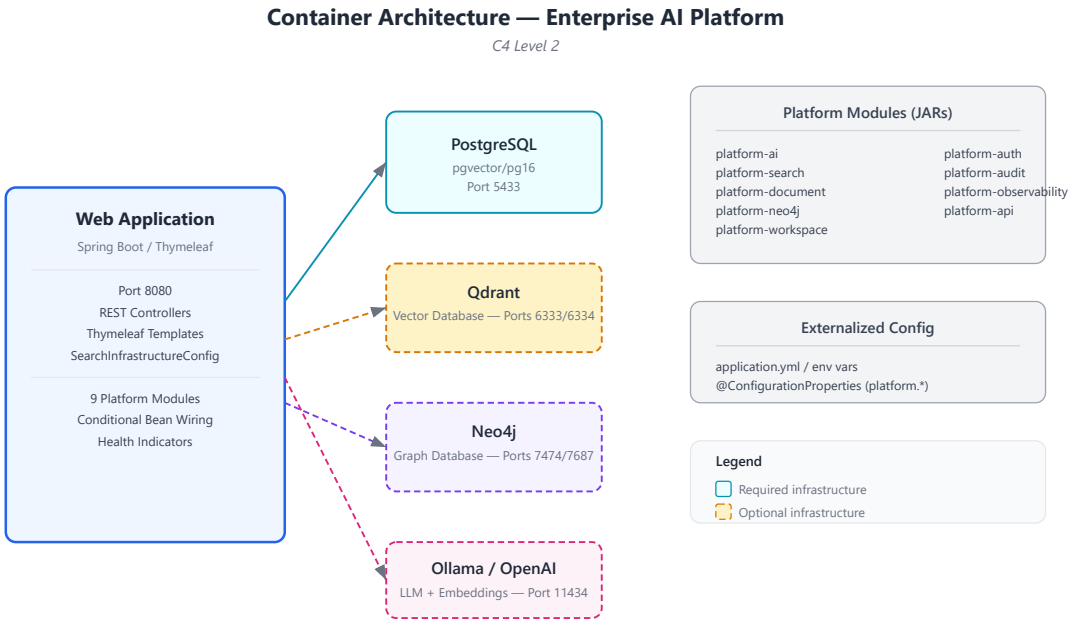

Figure 3.3. Runtime view: a single Spring Boot process communicates with its dependencies over standard protocols. The optional services are visually distinct — dashed borders, lighter fills — to communicate that the architecture expects them to sometimes be absent.

We chose Spring Boot 3.3 with Java 21. Not because it is the fastest framework, or the most lightweight, or the most novel. Because it is the most widely understood. A reference implementation should prioritize clarity over novelty. The Spring ecosystem — security, JPA, scheduling, Actuator, property binding — provides battle-tested infrastructure so the architecture can focus on AI concerns.

> ***Related:*** *ADR-001, ADR-002*

Now that the structure is clear, the natural next question is: what happens when a document enters the system?

---

4. The Document Journey

A document's journey through this platform is worth understanding in detail. It touches text extraction, semantic enrichment, chunking, embedding generation, and three separate persistence stores — all while the user who uploaded it has moved on to other things.

The key architectural decision was to run this asynchronously. Document processing takes seconds to minutes. HTTP request threads should not block that long. So upload creates a pending job and returns immediately. A scheduled worker picks it up.

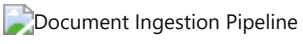

Figure 4.1. The ingestion pipeline runs asynchronously through a scheduled worker. Upload creates a PENDING job. The worker polls every 10 seconds, processes up to 10 jobs per cycle. Extraction, enrichment, chunking, and embedding run sequentially. Results land in PostgreSQL, Qdrant, and Neo4j.

Step 1 — Upload. An HTTP POST creates a `Document` entity and a `PENDING` ingestion job. The user gets an immediate response. The real work happens later.

Step 2 — Polling. A scheduled worker fires every 10 seconds, grabs the top 10 pending jobs, and feeds each through the ingestion processor. This is deliberately simple — no message queue, no priority system. A reference implementation should be easy to trace, not optimized for throughput.

Step 3 — Extraction. `TextExtractionService` delegates to format-specific parsers: Apache PDFBox for PDFs, Apache POI for DOCX, JSoup for HTML, no transformation for plain text.

Step 4 — Enrichment. Before chunking, the enrichment engine extracts entities — organizations, persons, technologies — plus concepts and relationships. If an LLM is available, it runs the `entity-extraction/v1` prompt for higher accuracy. If not, regex patterns provide a baseline. Results flow to Neo4j through the enrichment hook. The decision to enrich *before* chunking, not after, was discovered during testing: full document context produces better entity disambiguation than individual chunks ever could.

Step 5 — Chunking. The sentence-aware strategy splits at sentence boundaries: 1200-character target, 200-character minimum, 150-character overlap between consecutive chunks. Falls back gracefully when sentence boundaries are absent.

Step 6 — Embedding. If an embedding provider is configured — typically `nomic-embed-text` via Ollama — 768-dimensional vectors are generated. If not, chunks are stored keyword-only. The platform works either way.

Step 7 — Persistence. Chunks land in PostgreSQL through JPA, vectors in Qdrant through its REST API, and graph nodes in Neo4j through the Bolt protocol. Each store is optional beyond PostgreSQL. Each has its own fallback path.

One observation worth making explicit: this pipeline is sequential by design, not by limitation. Parallel enrichment and embedding would improve throughput. They would also make the code harder to trace and debug. The reference implementation prioritizes clarity. A production deployment would likely add parallelism — but the architecture doesn't prevent it.

Documents are now indexed, enriched, and graph-linked. The next question is: when a user asks a question, what happens?

---

5. How AI Is Wired In

Answering a question with AI is not one operation. It is twelve — and each one produces metadata that feeds the explainability trail. Understanding this pipeline is understanding the platform.

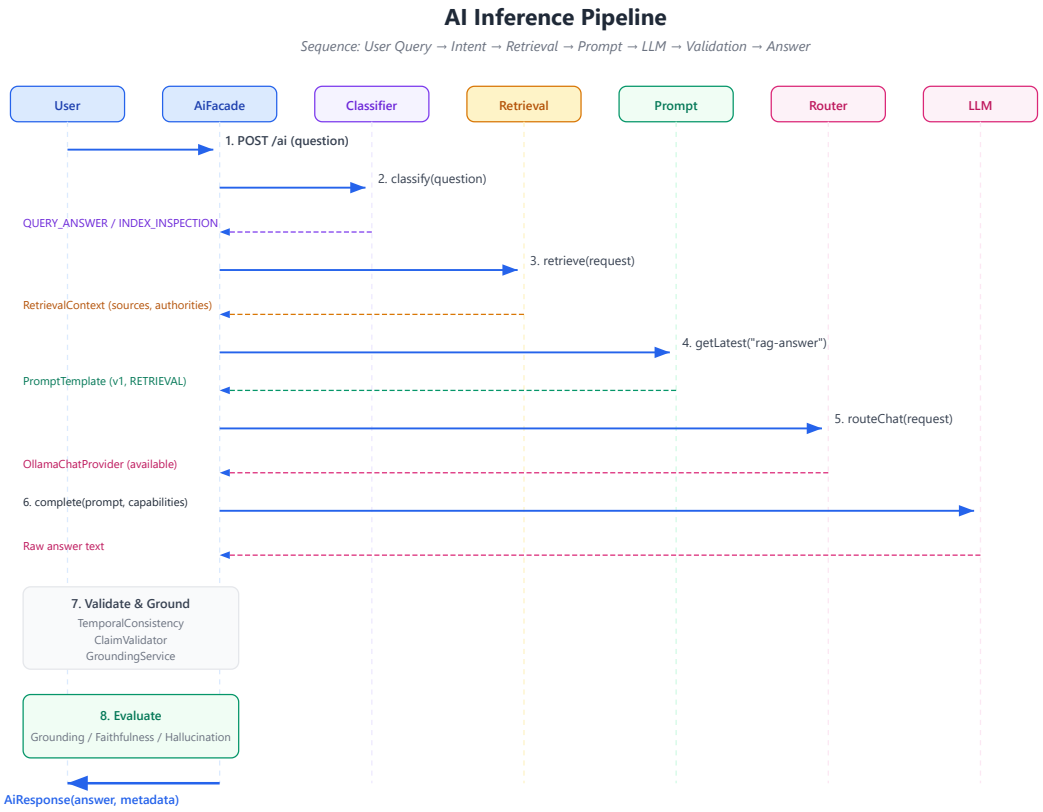

Figure 5.1. The twelve-step inference pipeline. Every step produces metadata. Steps 8-10 (validation, grounding, evaluation) run even when earlier steps produce warnings — the platform surfaces concerns rather than hiding them.

1. Intent classification. Before launching expensive retrieval, the system asks: what kind of query is this? An index inspection ("what documents are available?") routes to keyword-only — fast, exact, no vector search needed. A substantive question enters the full RAG pipeline.

2. Strategy selection. The classified intent maps to a retrieval strategy: keyword for exact lookups, hybrid for complex questions, semantic for conceptual exploration. The mapping is deterministic — same query always produces same strategy. This is deliberate. Non-deterministic routing is harder to debug, harder to audit, and the accuracy gain does not yet justify the complexity.

3. Retrieval. Search fans out to keyword, vector, and optionally graph sources. Results are fused through weighted linear combination and reranked through a two-stage process: score sort, then LLM cross-encoder if available.

4. Context assembly. Retrieved chunks, authority references, and analysis objectives are assembled into a structured prompt context.

5. Prompt building. The prompt registry resolves the latest `rag-answer` template. Variables are substituted. The rendered prompt includes the retrieved context and the user's question.

6. Provider selection. The provider router chooses which LLM backend to use — Ollama local, OpenAI cloud, or whichever provider is configured and available. Four-tier selection: model prefix, capability match, preference, first available.

7. LLM inference. The selected provider generates raw answer text.

8. Validation. Temporal consistency checker looks for hallucinated dates. Claim validator checks that assertions cite available sources. Warnings are appended to the answer — the platform surfaces concerns rather than hiding them.

9. Grounding. Sources are re-attributed. A multi-dimensional confidence profile is computed: source confidence, semantic confidence, structural confidence, completeness confidence, overall confidence.

10. Evaluation. Grounding score, faithfulness, hallucination indicators, answer relevance, context relevance. A pass/fail quality gate based on configurable thresholds.

11. Explainability. Every decision from steps 1-10 is captured in a structured result — traceable, auditable, deterministic. The `explain()` method produces a human-readable summary: "Intent: QUESTION_ANSWERING | Strategy: HYBRID | Provider: ollama | Model: qwen2.5:14b | Prompt: rag-answer/v1 | Sources: 15 | Duration: 245ms"

12. Response. The reasoned answer with full metadata is returned to the caller.

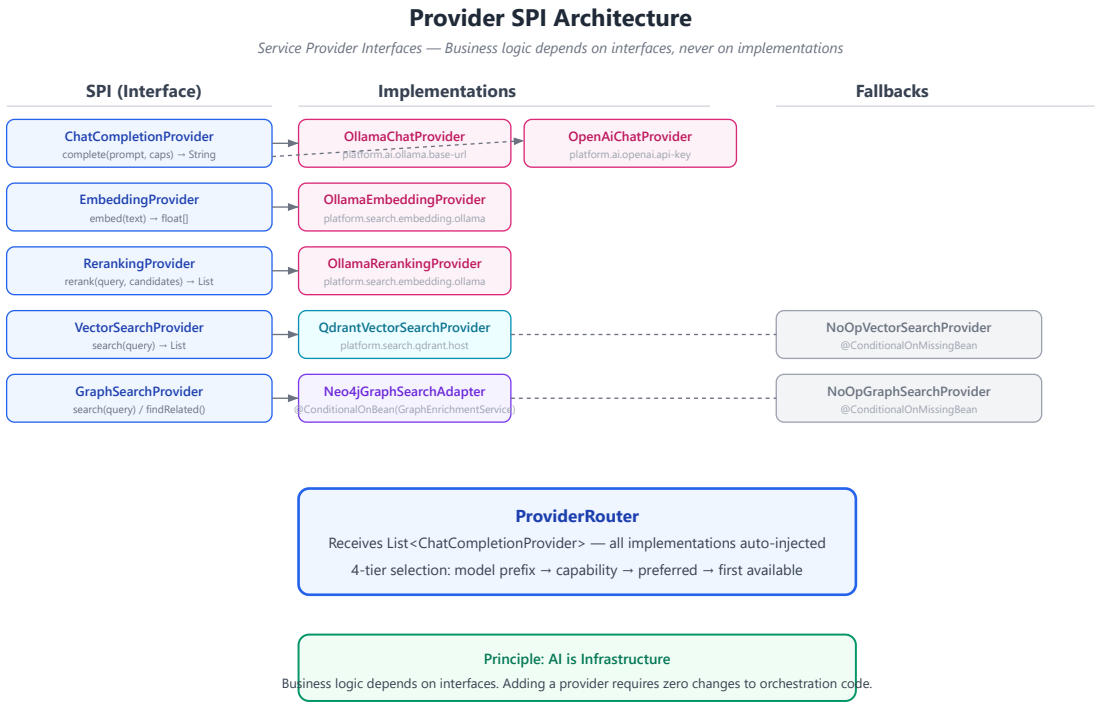

Figure 5.2. Five service provider interfaces decouple business logic from infrastructure. Each implementation activates conditionally based on configuration. NoOp fallbacks ensure the platform works without any optional service. Adding a provider means implementing the interface and adding `@Component` — zero changes to orchestration code.

The provider architecture embodies the most important principle in this platform: business logic depends on interfaces. Implementations depend on configuration. The two never meet in source code. This is not a novel pattern — it is the Strategy pattern, the Dependency Inversion Principle, ports and adapters — but applying it consistently to AI infrastructure, where most projects hardcode provider specifics, is what makes the architecture worth studying.

The inference pipeline answers questions. But the quality of those answers depends on something that happened hours earlier: enrichment.

---

6. Semantic Enrichment

Keyword search finds exact matches. Vector search finds semantically similar passages. Neither understands that "Acme Corporation" is an organization, or that "$45.2 million" relates to "revenue," or that "Dr. Sarah Chen" signed a document dated "January 15, 2026."

This structured understanding — who, what, when, how much — is what enables intelligent retrieval. Without it, a query like "What contracts did Acme sign in Q2?" must rely on the hope that relevant documents contain the exact words "Acme" and "Q2" near the word "contract."


Semantic Enrichment Pipeline

Figure 6.1. Enrichment runs after text extraction and before chunking. Full document context improves entity disambiguation. Results flow to Neo4j when available. When Neo4j is unavailable, enrichment still runs — only graph persistence is skipped.

The enrichment engine uses two extraction modes, selected automatically:

- **LLM-based:** When a chat provider is available, the `entity-extraction/v1` prompt extracts entities, concepts, and relationships with higher accuracy. The LLM understands context — it can distinguish "Apple the company" from "apple the fruit" based on surrounding text.
- **Regex fallback:** When no LLM is available, pattern matching catches organizations (based on suffixes like Inc., Corp., LLC), persons (Mr./Ms./Dr. prefixes), dates (ISO, US, European formats), and monetary amounts. The regex patterns are English-centric — a deliberate scope limitation — but the approach degrades rather than fails for other languages.

One design choice worth highlighting: every graph node carries provenance. Not as optional metadata — as required fields. Source document ID. Chunk ID. Extraction confidence. Extraction timestamp. Extraction model. Prompt version. Provider. Without provenance, a node claiming "Acme Corporation is an ORGANIZATION" could be a hallucination. With provenance, you can trace it to the exact document, chunk, model, and prompt that produced it. This makes the knowledge graph auditable. Not in theory — in practice, in every node, by construction.

> ***Design Rationale:*** *Why enrich during ingestion rather than query time? Because enrichment is expensive — LLM calls, regex passes over full documents — and query-time latency budgets are measured in milliseconds. Pre-computing enrichment during ingestion means retrieval is fast and the enrichment quality is known before any question is asked.*

Enrichment populates the graph. The next question is: how does that graph improve retrieval?

---

7. GraphRAG

Traditional RAG retrieves chunks by keyword and vector similarity. It works well when relevant documents share vocabulary or embedding proximity with the query. It works poorly when documents are semantically related but textually distant.

Consider two documents. One is a contract signed by Acme Corporation. The other is a financial report mentioning Acme's Q2 revenue. They share no keywords and their embeddings may sit far apart in vector space. But they are undeniably related — through the entity "Acme Corporation."

A knowledge graph captures this relationship explicitly. Traversing it reveals connections that keyword and vector search never would.

This was the motivation for GraphRAG. Not because graph traversal is theoretically elegant — though it is — but because it solves a real retrieval blind spot.

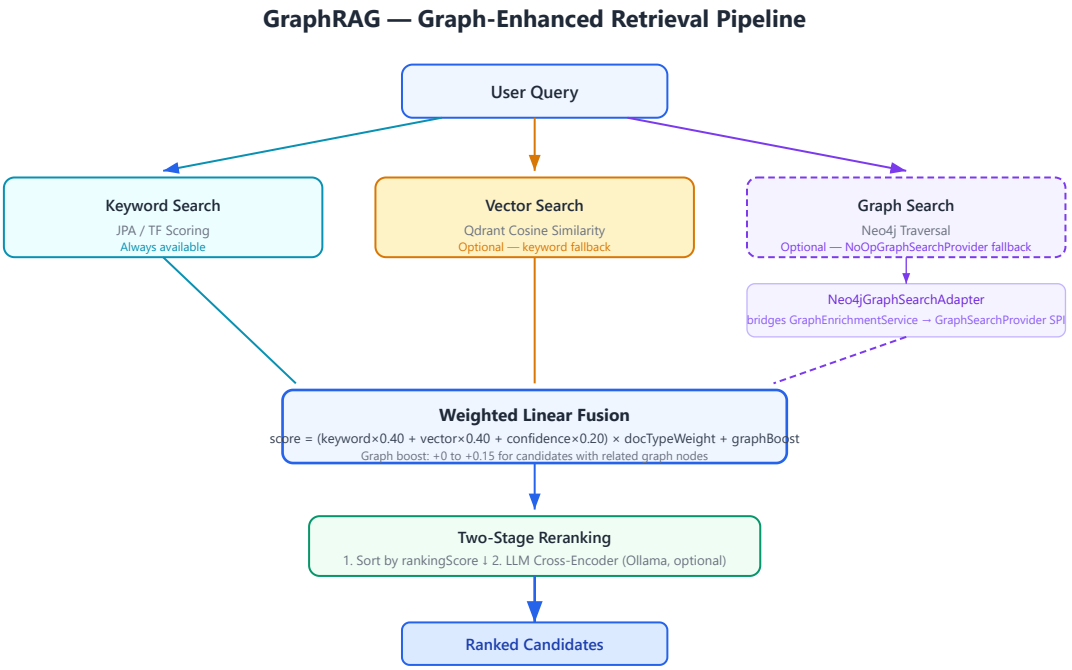

Figure 7.1. Three retrieval sources converge in weighted fusion. Notice the graph path is optional — dashed border, dashed lines. When Neo4j is unavailable, the system degrades to keyword + vector with no code changes. Graph results boost existing candidates; they don't compete with them.

The central design choice: **graph results boost, not replace.** The `combineWithGraph()` method adds up to 15% score increase for candidates with related graph nodes. A document that ranks #15 by keyword + vector could move to #5 because it mentions the same entities as a top-ranked document. Graph context nudges rankings; it doesn't override them.

The `Neo4jGraphSearchAdapter` lives in `platform-api` because it bridges two modules that should not depend on each other directly. The search module defines the `GraphSearchProvider` SPI. The Neo4j module provides graph services. The adapter — in the assembly module — connects them. This is ports and adapters in practice: the assembly layer wires together components that are unaware of each other's existence.

An honest admission: GraphRAG was documented as a feature for months before it was implemented. The architecture manual described Neo4j participating in retrieval. The code used `NoOpGraphSearchProvider` — a permanent fallback. Building the adapter closed that gap. The lesson is worth stating plainly: document what *is* built, not what *should be* built. Documentation that runs ahead of implementation creates confusion. Documentation that trails implementation is always accurate.

> ***Operational Note:*** *Graph search activates with* `SearchMode.GRAPH` *or* `SearchMode.HYBRID_GRAPH`*. When Neo4j is unavailable,* `NoOpGraphSearchProvider` *returns empty results. The fusion layer handles this transparently — keyword and vector results are unaffected.*

We have seen how enrichment populates the graph and how the graph improves retrieval. One more retrieval question remains: who decides *which* retrievers to run?

---

8. The Retrieval Decision Layer

Running every retriever for every query wastes resources. An index inspection query ("what documents are available?") does not need vector search. A conceptual question ("what capabilities does this platform have?") benefits from semantic similarity. A complex analytical question benefits from hybrid search with graph augmentation.

The solution is a decision layer — a component whose sole responsibility is choosing which retrievers to use based on what the query is trying to do.

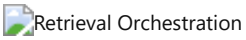
Retrieval Orchestration

Figure 8.1. Intent drives strategy. The mapping is deterministic — same query always produces same strategy. Every routing decision is logged in the trace log for auditability.

The `RetrievalOrchestrator` implements a simple but effective mapping:

Intent	Strategy	Rationale
Index inspection, corpus discovery	Keyword only	Exact match queries; vector search unnecessary
Question answering, analysis	Hybrid	Complex queries benefit from keyword + vector fusion
Document research, lookup	Semantic	Conceptual queries match better with vector similarity

The fusion algorithm uses weighted linear combination — not Reciprocal Rank Fusion. This is a deliberate naming choice. RRF requires rank positions from every retriever, which are not always available (graph search returns discovery results, not ranked lists). Weighted linear fusion works with raw scores and is simpler to explain.

```
score = (keyword×0.40 + vector×0.40 + confidence×0.20) × docTypeWeight + graphBoost
```

> *One surprising consequence of deterministic routing: it makes the system easier to debug than ML-based routing, but it also means query classification errors propagate deterministically. A misclassified query always takes the wrong path. The keyword-based classifier is simple and explainable, but it is not highly accurate. This is an explicit trade-off. If query classification accuracy becomes critical, the classifier can be replaced without touching the orchestrator — the architecture supports it.*

The orchestrator selects a strategy. The retrievers execute it. But what happens right before the LLM is called? That is where prompts enter the picture.

---

9. Prompts as Engineering Assets

Most projects treat prompts as strings — embedded in code, changed without versioning, auditable only through git blame. This works for prototypes. It fails for platforms where the prompt *is* the product — where a prompt change can alter answer quality across every query.

Prompts deserve the same engineering discipline as database schemas. Versioning. Categorization. Discoverability. Audit trails.

The prompt registry provides this. Every prompt has a qualified ID (`rag-answer/v1`), a category, declared variables, expected output type, supported models, recommended temperature, and usage examples. The `render()` method substitutes `{{variable}}` placeholders — simple, predictable, no template engine dependency.

Ten categories organize prompts by purpose: retrieval, summarization, extraction, classification, evaluation, reasoning, workflow, graph, search, system. The default registry seeds eight prompts. Production deployments would load from external resources.

> *One thing that became obvious only after implementing the registry: prompt versioning is not just for auditing. It enables A/B testing of prompt changes. When the `rag-answer` system instructions were modified between v1 and v2, evaluation scores could be compared directly — the same queries, the same retrieved context, different prompts. Without versioning, prompt changes are invisible and their effects are discovered in production.*

The registry is deliberately simple. No template engine. No inheritance. No dependency between prompts. Complexity in prompt management should be justified by need, not added preemptively.

---

10. Models as Infrastructure

Different models have different capabilities. GPT-4o supports vision and structured output. nomic-embed-text only does embeddings. qwen2.5:7b is fast and local but lacks vision. Business logic must not hardcode assumptions like "this model supports JSON."

The model capability registry describes every known model across eight capability dimensions plus quantitative metadata: context window, output tokens, latency estimates, cost estimates, recommended temperature, preferred use cases. Six models are seeded: four Ollama, two OpenAI.

The provider router uses this registry to make intelligent selection decisions through four tiers:

1. **Model prefix:** `openai:gpt-4o` routes to the OpenAI provider
2. **Capability match:** Request `STREAMING` → find a provider with a streaming-capable model
3. **Preferred provider:** Explicit caller preference
4. **First available:** First provider reporting healthy

The router receives `List<ChatCompletionProvider>` — Spring injects every implementation automatically. Adding a provider means implementing the interface and adding `@Component`. The router discovers it without code changes. This is the Strategy pattern applied to AI infrastructure — and it is worth studying because it is the pattern that enables the entire provider abstraction.

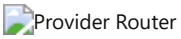
Provider Router

Figure 10.1. Four-tier provider selection. Each tier is logged. If all tiers fail, an exception is thrown — the platform never silently degrades to an unknown provider. The failure message tells you exactly which tiers were attempted and why each failed.

> *The architecture intentionally prevents business logic from selecting providers directly. Services request capabilities — streaming, JSON output, embeddings — and the router maps capabilities to providers. This indirection is the mechanism that makes adding new providers a zero-change operation for business logic.*

---

11. Workflows Without Hardcoding

Document intelligence involves multi-step processes. Upload leads to extraction, enrichment, chunking, indexing, review, and completion. Hardcoding these steps in a controller couples process logic to HTTP concerns and prevents reuse.

The workflow engine separates process definition from execution. A definition declares steps and transitions. An instance tracks current state. Five methods cover the API: `start`, `advance`, `previous`, `findInstance`, `listActive`.

Two workflows come pre-registered: `document-intelligence` (SETUP → INGESTION → ANALYSIS → REVIEW → COMPLETE) and `batch-ingestion` (INGEST → ENRICH → COMPLETE). Steps declare handler types — manual, automated, AI — as extension points. The current implementation is in-memory, suitable for a single-node reference. A database-backed store would be needed for production durability.

> *The architecture deliberately avoids BPMN engines or state machine frameworks. Five methods on an interface. Deterministic linear transitions. This is not the most powerful workflow engine. It is the simplest one that demonstrates the concept. A production system might adopt Camunda or Temporal — and the interface was designed so that substitution would not require changing the callers.*

---

12. The Audit Trail

If this platform generates an answer, it can explain itself. Not because someone added an "explain" button — because the orchestration layer records every decision as it happens. Intent classification. Strategy selection. Prompt template and version. Provider and model. Result counts from every retriever. Fusion method. Reranking status. Evaluation scores. A human-readable summary produced by `explain()`.

This is not a feature. It is the architecture. The metadata is generated during normal execution — there is no separate "explain this" pass, no post-hoc analysis, no second LLM call to generate explanations. Every code path produces metadata by construction.

> *One design principle guided this decision: explainability must have zero cost in the happy path. If explaining an answer requires additional computation, it will be disabled under load. By making explanation a byproduct of execution, it stays on by default, in every environment, forever.*

---

13. Observing What Happens

A platform running AI workloads needs observability designed for AI workloads. Not just request counts and error rates — but inference latencies broken down by provider and model, embedding batch durations, retrieval timings by strategy, enrichment entity counts, and evaluation score distributions.

The observability module provides Micrometer metrics for every AI operation. Timers with percentile histograms for inference, embedding, retrieval, graph traversal, enrichment, and ingestion. Gauges for provider availability. Summaries for evaluation scores. All exported to Prometheus at `/actuator/prometheus` and surfaced in health checks at `/actuator/health`.

> *The metrics infrastructure is defined and ready. It is not yet fully wired into every service — a deliberate prioritization decision. The metric definitions stabilize first. Full integration follows. This is how infrastructure should evolve: define the interface, validate it with a few integrations, then roll it out broadly. The alternative — wiring everything at once — produces metrics that nobody asked for and nobody uses.*

---

14. Security

Authentication uses JWT (HS256) with BCrypt-12 password hashing. Refresh tokens are hashed before storage — never stored in plaintext — and rotated on use. Both stateless API authentication (Bearer tokens) and session-based form login are supported.

Provider API keys are injected via `${ENV_VAR}` substitution — never hardcoded, never committed. User input enters the system through `{{question}}` variables in vetted prompt templates. Controller responses are HTML-escaped. The security model is standard, proven, and deliberately unremarkable. Innovation in security is a bug, not a feature.

---

15. How We Test This

The test suite is structured as executable architecture documentation. A Staff Engineer should understand the platform by reading the tests.

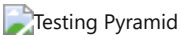
Testing Pyramid

Figure 15.1. Five testing layers from unit to browser. Default `mvn verify` runs everything except Playwright, which uses a separate profile. This keeps the primary build fast while still providing browser-level validation when needed.

Layer	Tests	What It Validates
Unit	44	Individual components — Prompt Registry, Provider Router, Model Registry
Integration	91	Subsystem interaction with real Spring context

Architecture	10	End-to-end behavior — enrichment, retrieval, evaluation
Resilience	9	Graceful degradation — provider fallback, graph unavailable
Contract	5	SPI stability — every provider must satisfy the abstract contract
Playwright	12	Browser user journeys (separate profile)

Tests describe behavior, not implementation. "Routes openai:gpt-4o to openai provider" — not "testProviderRouter." This naming convention is not cosmetic. It means a new engineer can read the test suite and understand what the platform does without reading the implementation.

---

16. Deployment

The platform starts with `docker compose up -d` for PostgreSQL and Qdrant. Add `--profile graph` for Neo4j. Then `mvn spring-boot:run -pl platform-api`. Configuration lives in `application.yml` with `${ENV_VAR:default}` substitution. Every custom property uses the `platform.*` namespace.

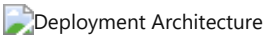
Deployment Architecture

Figure 16.1. Production deployment. Neo4j is in the `graph` Docker Compose profile. The platform starts with only PostgreSQL. Every additional service adds capability but is never required.

The deployment model is deliberately simple: one process, one configuration file, one command to start. A reference implementation should be easy to run on a development machine. Production deployments would add orchestration, secrets management, and monitoring — but those are deployment concerns, not platform concerns.

---

17. Extending the Platform

The platform was designed for extension. Four patterns cover the most common needs.

Adding a provider. Implement `ChatCompletionProvider`. Add `@Component` + `@ConditionalOnProperty`. Register model capabilities. The router discovers it automatically.

Adding a domain. Implement `DomainConfiguration`. Provide it as a Spring bean. Concepts, objectives, hierarchies, and instructions become available to the AI pipeline without core changes.

Adding a workflow. Define steps and transitions. Call `workflowEngine.start()`. The engine handles state.

Adding a prompt. Call `promptRegistry.register()`. The prompt is immediately versioned, categorized, and discoverable.

> *The goal was never to predict every extension point in advance. It was to establish a pattern — SPI, conditional activation, registry, injection — that makes extension feel natural rather than like working around the architecture.*

---

18. Performance Notes

This is a reference implementation, not a production deployment. It has not been benchmarked. What follows are architectural observations, not performance guarantees.

Chunking scales linearly with document length. Embedding generation is sequential — parallelizable via virtual threads when throughput matters. Weighted fusion is linear in total candidate count. Graph traversal is bounded at 2 hops. LLM inference timeout is configurable (default 120 seconds). Ingestion processes 10 documents per polling cycle — configurable, not optimized.

No caching layer exists. Each retrieval re-executes. This is appropriate for demonstrating the architecture. It would be the first thing to change in production.

---

19. The Trade-offs

These decisions were debated, documented, and accepted. They represent engineering judgment, not universal truth.

Decision	Chosen	Alternative	Why
Architecture	Modular monolith	Microservices	Compile-time boundaries sufficient; future extraction possible
Framework	Spring Boot 3.3	Quarkus	Ecosystem breadth; enterprise familiarity
Graph DB	Neo4j property graph	RDF / pgvector	Cypher for traversal; schema flexibility; mature driver
Fusion	Weighted linear	Reciprocal Rank Fusion	Heterogeneous retriever scores; simpler to explain
Prompts	Versioned registry	Inline strings	Audit trails; regression testing; A/B comparison
Enrichment	Dual-mode (LLM+regex)	LLM only	Works without AI infrastructure
Provider routing	4-tier deterministic	ML-based	Explainable; auditable; sufficient for current model count
UI testing	Playwright	Selenium	Modern API; auto-waits; better reliability

> *The thread connecting these decisions: prefer explicitness over magic. Prefer simplicity over power. Prefer what can be explained over what performs best on a benchmark. This is not always the right answer. It is the right answer for a reference implementation meant to be studied.*

Each decision has a dedicated ADR in Appendix A with deeper rationale.

---

20. What We Learned

Clean interfaces make domain-specific code easy to remove. The original version contained 55 German BGB paragraph references and 25+ legal keywords. They were removed without architectural changes — the interfaces had clean separation from domain logic. The lesson generalizes: invest in interface design. It pays off when requirements change.

Unused dependencies accumulate invisibly. Five JARs from Spring AI sat on the classpath for months with zero imports. A monthly `mvn dependency:analyze` costs nothing and prevents this.

Bean wiring tests are essential. The entire AI inference pipeline compiled and passed tests but was never injected. A controller showed retrieval results without generating LLM answers. A wiring verification test caught this. In Spring applications, test that your beans are actually used.

Document what is built, not what should be built. GraphRAG appeared in documentation months before implementation. The adapter that bridged the gap was built to close it. Now the code and documentation agree.

Prompt versioning is table stakes. What started as a simple registry became the foundation for deterministic behavior validation, A/B testing prompts, and audit trails.

Tests as documentation works. Evolving from wiring verification to behavioral testing made the test suite readable by architects who never touched the implementation.

---

21. Where This Goes Next

Done. Modular monolith, multi-provider AI, prompt and model registries, semantic enrichment, GraphRAG, retrieval orchestration, evaluation, explainability, workflow engine, observability, graceful degradation, DomainConfiguration SPI, 157 tests.

In progress. DomainConfiguration SPI exists but 8 services embed their configurations. Metrics component defined but not fully wired. Workflow engine has no PhaseHandler implementations.

Future. Dynamic model discovery from provider APIs. Resilience4j integration. OpenTelemetry tracing. Streaming LLM responses. Prompt A/B testing. Automated evaluation regression testing. Spring Modulith verification.

> *The implementation can change. The principles should remain. AI will evolve — new models, new providers, new retrieval techniques. But treating AI as infrastructure, degrading gracefully, explaining every decision, and extracting knowledge from documents rather than curating it manually — these principles are independent of any specific technology. They are what make this an architecture worth studying, not just a codebase worth reading.*

Appendix A — Architecture Decision Records (Overview)

Twenty Architecture Decision Records document every major engineering decision behind this platform. Each ADR follows the standard format: Context, Decision, Alternatives Considered, Consequences, Trade-offs, Future Evolution.

The complete ADR volume is available as a standalone document: **Architecture-Decision-Records.pdf**.

ADR	Title	Status	Category
001	Modular Monolith	Accepted	Architecture
002	Spring Boot 3.3 as Application Framework	Accepted	Technology
003	Provider Abstraction via SPI	Accepted	AI Infrastructure
004	Provider Router for Intelligent Model Selection	Accepted	AI Infrastructure
005	Prompt Registry for Versioned Prompt Management	Accepted	AI Infrastructure
006	Model Capability Registry	Accepted	AI Infrastructure
007	Semantic Enrichment Engine	Accepted	Knowledge
008	GraphRAG — Graph-Enhanced Retrieval	Accepted	Knowledge
009	Neo4j as Knowledge Graph Persistence	Accepted	Knowledge
010	Qdrant as Vector Database	Accepted	Knowledge
011	Retrieval Orchestration with Intent-Based Strategy	Accepted	Knowledge
012	Explainability by Default	Accepted	Quality
013	Evaluation Engine	Accepted	Quality
014	Workflow Engine	Accepted	Extensibility
015	AI Observability with Micrometer	Accepted	Operations
016	Graceful Degradation Over Hard Failures	Accepted	Operations
017	DomainConfiguration for Multi-Domain AI	Accepted	Extensibility

018	Provenance-Aware Knowledge Graph	Accepted	Knowledge
019	Multi-Layer Testing Strategy	Accepted	Quality
020	Enterprise AI Platform Design Philosophy	Accepted	Governance

---

Appendix B — Glossary

Appendix B — Glossary

Capability — A feature an AI model supports: streaming, vision, JSON mode, embeddings, tool calling.

ChatCompletionProvider — The SPI for LLM backends. Every provider (Ollama, OpenAI) implements this interface.

Chunk — A segment of document text (~1200 characters) with sentence-boundary awareness and overlap, stored and indexed for retrieval.

Citation Coverage — The fraction of AI answer claims that cite a retrieved source.

Context Window — The maximum tokens a model can process in a single request. Ranges from 8K to 200K+.

DomainConfiguration — SPI for domain-specific rules: concepts, objectives, hierarchies, instructions.

Embedding — A 768-dimensional vector representation of text, generated by an embedding model.

Enrichment — Automatic extraction of entities, concepts, and relationships from documents during ingestion.

Evaluation — Automated quality assessment: grounding, faithfulness, hallucination indicators, relevance.

Explainability — Metadata describing how an inference was produced, generated as a byproduct of execution.

Faithfulness — How factually accurate an answer is relative to its source documents. Inverse of hallucination.

Fusion — Combining retrieval results from multiple sources into a single ranked list.

GraphRAG — RAG enhanced with knowledge graph traversal. Graph results boost, not replace.

Grounding — The degree to which an answer is supported by retrieved evidence.

Hallucination — AI-generated text not supported by source documents.

Hybrid Search — Retrieval combining keyword, vector, and optionally graph search.

Inference — The process of an LLM generating text from a prompt.

Intent — The classified purpose of a query: question answering, document lookup, index inspection.

Knowledge Graph — A network of entities, concepts, and relationships extracted from documents.

NodeProvenance — Metadata linking a graph element to its source document, chunk, model, prompt version, and timestamp.

Prompt Registry — Versioned store of prompt templates organized by category.

Provider — An AI backend implementing a platform SPI.

Provider Router — Component that selects which provider to use. 4-tier deterministic selection.

Qdrant — Vector database for high-dimensional similarity search. Optional infrastructure.

RAG — Retrieval-Augmented Generation. Grounding LLM responses in retrieved documents.

Registry — A queryable, versioned store of known entities (prompts, models, capabilities).

Reranking — Reordering candidates: score sort followed by LLM cross-encoder if available.

Retrieval Strategy — Which retrievers to use, selected based on classified query intent.

Semantic Enrichment — Extraction of structured knowledge from text during ingestion.

SPI — Service Provider Interface. A pluggable contract for extending the platform.

Streaming — Real-time token-by-token LLM output. Not yet implemented.

Structured Output — LLM responses constrained to a format like JSON. Supported by GPT-4o and Llama 3.2.

Token — A unit of text processed by an LLM (~0.75 words in English).

Vector — A numerical representation of text meaning, 768 dimensions in this platform.

Vector Database — Database optimized for high-dimensional similarity queries.

Workflow — A configurable multi-step process with defined state transitions.

Workflow Engine — Reusable component for executing workflows.

Appendix C — References

Appendix C — References

Architecture & Engineering

- **Spring Boot 3.3 Reference** — The platform's runtime framework
- **Martin Kleppmann, *Designing Data-Intensive Applications*** — Foundational thinking on data systems
- **Martin Fowler, *Patterns of Enterprise Application Architecture*** — Patterns for modular design
- **Simon Brown, C4 Model** — Architecture visualization approach

AI & Retrieval

- **Lewis et al. (2020), *RAG for Knowledge-Intensive NLP Tasks*** — Foundational RAG paper
- **Microsoft Research (2024), *GraphRAG: From Local to Global*** — Graph-enhanced retrieval
- **Neo4j Graph Data Science** — Graph algorithms for centrality and community detection
- **Qdrant Documentation** — Vector search with payload filtering

Operations

- **Micrometer** — JVM metrics instrumentation
- **Prometheus** — Metrics collection and alerting
- **Playwright** — Cross-browser automation
- **OpenTelemetry** — Distributed tracing standard (future)

Internal

- TESTING.md
- Diagram Inventory

ADR-001-modular-monolith

ADR-001 — Modular Monolith Architecture

STATUS

Accepted. Implemented in the Maven module structure at `pom.xml`.

CONTEXT

The Enterprise AI Platform must support multiple AI-powered applications (document intelligence, contract analysis, financial review, compliance, etc.) while remaining deployable as a single process. The platform must demonstrate clean separation of concerns without the operational complexity of microservices.

DECISION

Adopt a **Modular Monolith** architecture with 9 Maven modules:

```
platform-audit       — Immutable audit log, correlation IDs
platform-auth        — JWT authentication, refresh token rotation
platform-document    — Document lifecycle, ingestion pipeline
platform-search      — Hybrid search (keyword + vector + graph)
platform-ai          — RAG orchestration, enrichment, evaluation, registry
platform-neo4j       — Auto-generated knowledge graph with provenance
platform-workspace   — Multi-phase workflow wizard
platform-observability — Micrometer metrics, health indicators
platform-api         — REST + Thymeleaf controllers, assembly
```

Module boundaries follow **dependency inversion**: higher-level modules depend on lower-level APIs, never the reverse. `platform-api` is the assembly module that wires everything together.

ALTERNATIVES CONSIDERED

- **Microservices**: Rejected. The platform demonstrates architecture without distributed systems complexity. Microservices add network boundaries, eventual consistency challenges, and deployment overhead that don't benefit a reference implementation.
- **Single JAR without modules**: Rejected. A flat structure would not enforce dependency direction or demonstrate separation of concerns.
- **OSGi modules**: Rejected. Adds runtime modularity complexity. Maven's compile-time enforcement is sufficient.

CONSEQUENCES

- **Clear dependency direction**: `api → ai → search → document → audit` (audit is a leaf dependency)
- **Compile-time enforcement**: Modules cannot accidentally depend on each other
- **Single deployable**: One `platform-api` Spring Boot application with all modules on the classpath
- **Test isolation**: Each module can be tested independently

TRADE-OFFS

- Module boundaries are enforced only at compile time, not runtime
- Adding a new module requires POM maintenance
- Some modules (audit) are cross-cutting dependencies of many others

FUTURE EVOLUTION

- Modules may split if responsibilities grow too large (e.g., `platform-ai` could become `platform-ai-core` + `platform-ai-providers`)
- Spring Modulith could be introduced for runtime module verification
- The architecture deliberately supports future extraction of modules into microservices if needed

See also: [[ADR-002]], [[ADR-019]], [[ADR-020]]

ADR-002-spring-boot

ADR-002 — Spring Boot 3.3 as Application Framework

STATUS

Accepted. Implemented in `pom.xml` at `spring-boot-starter-parent:3.3.5`.

CONTEXT

The platform must be recognizable to experienced Java engineers, leverage mature ecosystems for security, persistence, and observability, and remain maintainable by teams familiar with enterprise Java.

DECISION

Use **Spring Boot 3.3** with Java 21 as the application framework. All modules depend on `spring-boot-starter-parent` BOM for version management. Key Spring integrations:

- `spring-boot-starter-web` — REST controllers
- `spring-boot-starter-security` — JWT authentication via Nimbus JOSE
- `spring-boot-starter-data-jpa` — PostgreSQL/H2 persistence
- `spring-boot-starter-actuator` — Health endpoints, metrics
- `spring-boot-starter-thymeleaf` — Server-rendered UI pages
- `spring-boot-starter-validation` — Request validation
- `@ConfigurationProperties` — Type-safe configuration binding
- `@ConditionalOnProperty` / `@ConditionalOnBean` — Provider activation

ALTERNATIVES CONSIDERED

- **Quarkus**: Rejected. Smaller ecosystem, less familiar to enterprise teams, fewer library integrations.
- **Micronaut**: Rejected. Strong compile-time DI but smaller community and fewer reference architectures.
- **Plain Java with manual DI**: Rejected. Would require reimplementing what Spring provides (security, JPA, scheduling, Actuator, property binding).
- **Spring Boot 2.x**: Rejected. Java 21 virtual threads require Spring Boot 3.x.

CONSEQUENCES

- **Rich ecosystem**: Spring Security, Spring Data JPA, Spring Actuator all integrated out of the box
- **Virtual threads**: Java 21 + Spring Boot 3.3 enable `spring.threads.virtual.enabled`
- **Conditional beans**: `@ConditionalOnProperty` enables graceful degradation for optional infrastructure
- **Configuration**: `@ConfigurationProperties` with `platform.*` prefix ensures consistent, type-safe config

TRADE-OFFS

- Startup time is slower than compile-time DI frameworks
- Spring's "magic" can obscure wiring for newcomers (mitigated by explicit `@Bean` definitions in `SearchInfrastructureConfig`)
- Memory footprint is larger than minimalist frameworks

FUTURE EVOLUTION

- Spring Modulith could add runtime module verification
- Spring AI could replace custom provider HTTP clients when mature
- GraalVM native image compilation could reduce startup time for serverless deployments

See also: [[ADR-001]], [[ADR-003]], [[ADR-015]]

ADR-003-provider-abstraction

ADR-003 — Provider Abstraction via Service Provider Interface (SPI)

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/api/ChatCompletionProvider.java` and related interfaces.

CONTEXT

AI backends (Ollama, OpenAI, Anthropic, Gemini) expose different APIs, authentication schemes, and request/response formats. Business logic must not depend on any specific provider implementation. Adding a new provider should require zero changes to orchestration code.

DECISION

Define **SPI interfaces** for every AI capability. Implementations are conditionally activated based on configuration:

Interface	Implementations	Activation
`ChatCompletionProvider`	`OllamaChatProvider`, `OpenAiChatProvider`	`@ConditionalOnProperty`
`EmbeddingProvider`	`OllamaEmbeddingProvider`	`@ConditionalOnProperty`

`RerankingProvider`	`OllamaRerankingProvider`	`@ConditionalOnProperty`
`VectorSearchProvider`	`QdrantVectorSearchProvider`, `NoOpVectorSearchProvider`	Conditional on host
`GraphSearchProvider`	`Neo4jGraphSearchAdapter`, `NoOpGraphSearchProvider`	Conditional on Neo4j
`EvaluationService`	`DefaultEvaluationService`	Always active

The `ProviderRouter` (`DefaultProviderRouter`) selects a provider using a 4-tier strategy:

1. Model name prefix (`openai:gpt-4o` → openai)
2. Capability match (streaming → capable provider)
3. Preferred provider
4. First available fallback

Business services depend on interfaces, never on concrete implementations.

ALTERNATIVES CONSIDERED

- **Hardcoded provider selection in business logic**: Rejected. Would couple orchestration to specific providers and make adding new providers expensive.
- **Spring AI abstraction**: Rejected. At implementation time, Spring AI 1.0.0 provided limited control over provider selection and lacked the capability registry concept. The dependency was removed during build audit.
- **Factory pattern without SPI**: Rejected. An SPI with conditional beans is more idiomatic in Spring and supports auto-discovery.

CONSEQUENCES

- **Provider-agnostic business logic**: `AiService` depends on `ChatCompletionProvider` , not Ollama or OpenAI
- **Conditional activation**: Providers activate only when their configuration is present
- **No-code provider addition**: Implement `ChatCompletionProvider` + `@Component` + `@ConditionalOnProperty`
- **Graceful degradation**: When no provider is available, `ProviderRouter` throws a clear exception

TRADE-OFFS

- Each provider implements its own HTTP client (no shared HTTP infrastructure)
- Provider implementations must handle their own error translation
- Capability discovery is static (seeded in `DefaultModelCapabilityRegistry`) rather than dynamic

FUTURE EVOLUTION

- Dynamic capability discovery via provider API introspection
- Shared HTTP client infrastructure with configurable retry/timeout
- Provider cost and latency metadata for intelligent routing

See also: [[ADR-004]], [[ADR-006]], [[ADR-016]]

ADR-004-provider-router

ADR-004 — Provider Router for Intelligent Model Selection

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultProviderRouter.java` .

CONTEXT

With multiple AI providers available (Ollama local, OpenAI cloud), the platform must select the appropriate provider for each inference request. Selection criteria include model name, required capabilities, user preference, and provider availability.

DECISION

Implement a **Provider Router** (`ProviderRouter` interface) with a deterministic 4-tier selection strategy:

1. **Model prefix routing**: `openai:gpt-4o` routes to the OpenAI provider, `ollama:qwen2.5` routes to Ollama

2. **Capability-based selection**: If a specific capability is requested (e.g., `STREAMING`), the router selects a provider whose models support that capability, as defined in the `ModelCapabilityRegistry`
3. **Preferred provider**: If the caller specifies a preferred provider, it is selected if available
4. **First available fallback**: The first provider reporting `isAvailable() == true` is selected

If no provider is available, the router throws `IllegalStateException` with a clear message.

Business services call `router.routeChat(InferenceRequest)` — they request capabilities, not specific providers.

ALTERNATIVES CONSIDERED

- **Direct injection of a single provider**: Rejected. Would make multi-provider setups impossible and require code changes to switch providers.
- **Random/round-robin selection**: Rejected. Non-deterministic behavior is harder to debug and audit.
- **Cost/latency-aware routing**: Deferred to future evolution. The router architecture supports adding routing dimensions.

CONSEQUENCES

- **Deterministic routing**: Same input always produces same routing decision
- **Auditable**: Every routing decision is logged (`log.debug("Routed '{}' to provider '{}' by model prefix", ...)`)
- **Extensible**: Adding a routing dimension means adding a strategy tier, not rewriting the router
- **Clear failure mode**: Unavailable providers are skipped with explicit fallthrough

TRADE-OFFS

- Static capability registry requires manual updates when new models are added
- No runtime performance tracking for latency/cost-based routing
- Model prefix convention (`provider:model`) is a convention, not enforced by types

FUTURE EVOLUTION

- Cost-aware routing using `ModelCapability.estimatedCostPer1kTokens()`
- Latency-aware routing using `ModelCapability.estimatedLatencyMs()`
- Region-aware routing for multi-region deployments
- Dynamic capability discovery from provider APIs

See also: [[ADR-003]], [[ADR-005]], [[ADR-006]]

ADR-005-prompt-registry

ADR-005 — Prompt Registry for Versioned Prompt Management

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultPromptRegistry.java`.

CONTEXT

AI prompts are a critical platform asset. Without versioning, prompt changes cannot be tracked, audited, or rolled back. Without a registry, prompts are scattered across Java string constants, making them impossible to discover or manage.

DECISION

Implement a **Prompt Registry** (`PromptRegistry` interface) that stores versioned prompt templates with metadata:

Feature	Implementation
Versioning	`PromptTemplate.id` + `PromptTemplate.version` → qualified ID `"rag-answer/v1"`
Categories	`PromptTemplate.Category`: RETRIEVAL, SUMMARIZATION, EXTRACTION, CLASSIFICATION, EVALUATION, REASONING, WORKFLOW, GRAPH, SEARCH, SYSTEM
Variables	`{{variable}}` template substitution via `render(Map<String, String>)`

Metadata	`expectedOutputType`, `supportedModels`, `recommendedTemperature`, `examples`
Discovery	`findByCategory(Category)`, `listPromptIds()`, `getLatest(String)`

The default registry (`DefaultPromptRegistry`) seeds 8 prompts across 6 categories. Prompts are registered programmatically; in production, they would be loaded from YAML/JSON resources.

Every inference records which prompt was used (`RetrievalOrchestrationResult.promptTemplateId()`), enabling full reproducibility and audit trails.

ALTERNATIVES CONSIDERED

- **Hardcoded string constants**: Rejected. No versioning, no discovery, no audit trail.
- **Database-backed prompt store**: Deferred. Adds infrastructure dependency. In-memory registry with resource loading is sufficient for a reference implementation.
- **External prompt management service**: Rejected. Over-engineered for a modular monolith.

CONSEQUENCES

- **Reproducibility**: Every inference records `promptTemplateId` + `promptTemplateVersion`
- **Discoverability**: `findByCategory()` enables prompt inventory
- **Safe evolution**: New prompt versions can be registered without deleting old ones
- **Regression testing**: Prompts are identifiable assets that can be tested

TRADE-OFFS

- In-memory storage means prompts reset on restart (acceptable for a reference implementation)
- No prompt validation at registration time (variables are not checked against template)
- Rendering uses simple string substitution, not a template engine (deliberate simplicity)

FUTURE EVOLUTION

- YAML/JSON resource loading for external prompt definitions
- Prompt validation: verify all declared variables are used and all template variables are declared
- Prompt A/B testing: serve different versions to different users
- Prompt migration tooling for bulk updates

See also: [[ADR-003]], [[ADR-004]], [[ADR-012]]

ADR-006-model-capability-registry

ADR-006 — Model Capability Registry

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultModelCapabilityRegistry.java` .

CONTEXT

Different AI models have different capabilities (streaming, vision, JSON mode, tool calling, embeddings). Business logic should not hardcode assumptions like "model X supports JSON." Instead, capabilities should be queryable through a central registry, enabling the `ProviderRouter` to make intelligent selection decisions.

DECISION

Implement a **Model Capability Registry** (`ModelCapabilityRegistry` interface) that describes every available model's capabilities:

Capability	Examples
`supportsStreaming`	qwen2.5:7b (true), nomic-embed-text (false)
`supportsVision`	gpt-4o (true)
`supportsJson` / `supportsToolCalling`	gpt-4o, llama3.2 (true)

`supportsEmbeddings`	nomic-embed-text (true)
`supportsStructuredOutput`	gpt-4o (true)
`maxContextWindow`	128K (gpt-4o), 32K (qwen2.5)
`maxOutputTokens`	16384 (gpt-4o)
`estimatedLatencyMs`	500 (qwen2.5 local), 1200 (gpt-4o cloud)
`estimatedCostPer1kTokens`	$5.00 (gpt-4o), $0.00 (local Ollama)
`preferredUseCases`	"rag", "analysis", "summarization"

The registry supports querying by:

- Exact model name: `get("gpt-4o")`
- Provider: `findByProvider("ollama")`
- Capability: `findByCapability(EMBEDDING)`
- Provider + capability: `findByProviderAndCapability("ollama", STREAMING)`

The `DefaultModelCapabilityRegistry` seeds with 6 known models (4 Ollama, 2 OpenAI). The `ProviderRouter` uses the registry for capability-based routing decisions.

ALTERNATIVES CONSIDERED

- **Hardcoded if/else chains**: Rejected. Unmaintainable with growing model lists.
- **Runtime model discovery via API**: Deferred. Provider APIs (Ollama `/api/tags`, OpenAI `/models`) could populate the registry dynamically, but add latency and failure modes.
- **No registry — trust the provider**: Rejected. The platform needs to know capabilities before calling a model (e.g., "does this model support JSON mode?").

CONSEQUENCES

- **Capability-aware routing**: `ProviderRouter` selects models based on required capabilities
- **Static seed data**: Registry is populated at startup with known models; new models require code changes
- **Query interface**: Rich query API enables future use cases (e.g., "find the cheapest model that supports vision")

TRADE-OFFS

- Static data can become stale as providers add new models
- Estimated costs and latencies are approximations, not real-time measurements
- No runtime validation that the model actually supports claimed capabilities

FUTURE EVOLUTION

- Dynamic discovery from provider `/models` endpoints
- Runtime capability verification (send test prompts to verify claims)
- User-provided model registrations via configuration
- Integration with provider cost APIs for real-time pricing

See also: [[ADR-003]], [[ADR-004]], [[ADR-005]]

ADR-007-semantic-enrichment

ADR-007 — Semantic Enrichment Engine

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultEnrichmentService.java` and `platform-api/src/main/java/com/cognitera/platform/api/ingestion/EnrichmentHook.java`.

CONTEXT

Documents contain unstructured information. To enable intelligent retrieval, the platform must extract structured knowledge — entities, concepts, and relationships — automatically during ingestion. Users should never manually populate knowledge bases.

DECISION

Implement an automatic **Semantic Enrichment Engine** that runs as part of the document ingestion pipeline:

```
Upload → Text Extraction → Enrichment → Chunking → Embedding → PostgreSQL + Qdrant
                                          ↓
                                    Neo4j Graph
```

The enrichment pipeline uses a dual-mode extraction strategy:

1. **LLM-based**: When a `ChatCompletionProvider` is available, prompts the LLM with `entity-extraction/v1` template for high-quality structured extraction
2. **Regex fallback**: When no LLM is available, uses regex patterns for known entity types (ORGANIZATION, PERSON, DATE, MONEY)

Results are bridged to Neo4j via `EnrichmentHook`, which converts `EnrichmentContext` to `GraphNode` / `GraphRelationship` objects and persists them through `GraphEnrichmentService`.

Entities carry **provenance**: `sourceDocumentId`, `chunkId`, `extractionConfidence`, `extractionTimestamp`, `extractionModel`, `promptVersion`, `provider`.

ALTERNATIVES CONSIDERED

- **Manual knowledge entry (old platform-knowledge module)**: Rejected. Users should not manually curate knowledge. The document corpus is the knowledge source. The old `platform-knowledge` module was removed in Phase 1.
- **LLM-only enrichment**: Rejected. Would fail when no LLM is available. Regex fallback ensures basic enrichment always works.
- **No enrichment**: Rejected. Without enrichment, retrieval is purely lexical/semantic with no structured understanding.

CONSEQUENCES

- **Automatic knowledge extraction**: Entities, concepts, and relationships are extracted during ingestion without user intervention
- **Graceful degradation**: Regex fallback when LLM is unavailable
- **Provenance**: Every graph node links back to its source document and extraction method
- **Neo4j population**: The knowledge graph is auto-generated, never manually curated

TRADE-OFFS

- Regex patterns are less accurate than LLM extraction
- The regex patterns are English-centric and need extension for multilingual documents
- Enrichment adds latency to the ingestion pipeline

FUTURE EVOLUTION

- Multilingual entity extraction patterns
- Domain-specific enrichment via `DomainConfiguration`
- Confidence threshold configuration for entity filtering
- Parallel enrichment for large documents
- Integration with external NER services

See also: [[ADR-008]], [[ADR-009]], [[ADR-017]], [[ADR-018]]

ADR-008-graphrag

ADR-008 — GraphRAG — Graph-Enhanced Retrieval

STATUS

Accepted. Implemented in `Neo4jGraphSearchAdapter` bridging `GraphEnrichmentService` to `GraphSearchProvider`, with integration in `DefaultHybridRetrievalService`.

CONTEXT

Traditional RAG retrieves chunks via keyword and vector similarity. However, semantically related documents may not share keywords or embedding proximity. A knowledge graph connecting entities, concepts, and documents enables traversal-based discovery that complements keyword and vector retrieval.

DECISION

Implement **GraphRAG** — graph-enhanced retrieval — as an optional third retrieval source alongside keyword and vector:

```
SearchQuery
  ├── KeywordSearchProvider.search()  → keywordResults
  ├── VectorSearchProvider.search()   → vectorResults
  ├── GraphSearchProvider.search()    → graphResults (optional)
  └── merge(keyword, vector, graph)   → fused candidates
       ├── combine(): weighted linear fusion (k×0.40 + v×0.40 + c×0.20)
       └── combineWithGraph(): graph boost (existing + graph×0.15)
  → Reranking → LLM
```

Graph results **boost** existing candidates rather than replacing them. The `combineWithGraph()` method adds up to 15% score increase when graph traversal finds related nodes.

Graph retrieval is **optional**: `SearchMode.GRAPH` and `SearchMode.HYBRID_GRAPH` activate it. When Neo4j is unavailable, `NoOpGraphSearchProvider` returns empty results and retrieval continues with keyword + vector.

ALTERNATIVES CONSIDERED

- **Graph-only retrieval**: Rejected. Graph traversal is useful for discovery but less precise for exact match queries.
- **Graph as primary source with keyword/vector as secondary**: Rejected. Documents without graph entities would be invisible.
- **No graph retrieval**: Rejected. The enrichment engine already populates Neo4j; not using it for retrieval wastes the enrichment investment.

CONSEQUENCES

- **Three-source fusion**: Keyword, vector, and graph results are merged in a single pipeline
- **Graceful degradation**: Graph retrieval is optional; platform works without Neo4j
- **Graph boost**: Related entities and concepts boost document scores by up to 15%
- **Explainability**: Graph participation is tracked in retrieval metadata

TRADE-OFFS

- Graph search uses simple entity name matching from the query (not embedded query → nearest neighbor in graph embedding space)
- The `Neo4jGraphSearchAdapter` creates synthetic `ChunkReference` objects for graph results, which have lower fidelity than real chunk references
- Graph boost of 15% is fixed, not calibrated per domain

FUTURE EVOLUTION

- Graph embedding models for semantic graph traversal
- Configurable graph boost factors per domain
- Multi-hop reasoning via graph traversal patterns
- Graph-native reranking using centrality metrics

See also: [[ADR-007]], [[ADR-009]], [[ADR-011]]

ADR-009-neo4j

ADR-009 — Neo4j as Knowledge Graph Persistence

STATUS

Accepted. Implemented in `platform-neo4j/src/main/java/com/cognitera/platform/neo4j/service/GraphEnrichmentService.java`.

CONTEXT

The platform needs to store structured knowledge extracted from documents — entities, concepts, and their relationships. This data is inherently graph-shaped: entities relate to documents, concepts relate to entities, documents cite other documents. A relational database would require complex recursive CTEs for graph traversal.

DECISION

Use **Neo4j** as the persistence layer for the automatically-generated knowledge graph. Neo4j is **not** a general-purpose database in this architecture — it stores only semantic enrichment output:

Node Type	Example
`DOCUMENT`	Uploaded PDF, DOCX, TXT
`ENTITY`	Organization, Person, Technology
`CONCEPT`	Temporal concepts, financial amounts, topics

Relationship	Example
`MENTIONS`	Document → Entity
`RELATED_TO`	Document → Concept
`BELONGS_TO`	Entity → Concept
`REFERENCES`	Document → Document

The graph is **auto-generated during ingestion** — never manually curated. The old `platform-knowledge` module (manual CRUD knowledge base) was removed in Phase 1.

Neo4j is **optional**: the `graph` profile in `docker-compose.yml` and `@ConditionalOnProperty(name = "platform.neo4j.uri")` ensure the platform starts without it.

ALTERNATIVES CONSIDERED

- **PostgreSQL with recursive CTEs**: Rejected. Graph traversal queries become complex and slow beyond 2-3 hops. Cypher is purpose-built for graph traversal.
- **Property graph in application memory**: Rejected. Does not persist across restarts; cannot scale beyond small corpora.
- **RDF triple store**: Rejected. Adds complexity (SPARQL, ontology management) without clear benefit over labeled property graphs for this use case.
- **No graph database**: Rejected. The enrichment engine produces graph-shaped data; storing it relationally would violate the "right tool for the data shape" principle.

CONSEQUENCES

- **Native graph traversal**: `GraphEnrichmentService.traverse(seedIds, maxDepth)` uses Cypher for multi-hop queries
- **Optional infrastructure**: Platform works without Neo4j
- **Provenance**: Every node carries `NodeProvenance` linking back to source document and extraction method
- **Auto-generated**: Graph population happens during ingestion, not through user interaction

TRADE-OFFS

- Adds infrastructure dependency when GraphRAG is desired
- Neo4j Community Edition has single-database limitation
- Graph schema is implicit (defined by code) rather than explicit (defined by constraints)

FUTURE EVOLUTION

- Graph embedding generation in Neo4j (GDS library)
- Cypher query templates for common traversal patterns
- Graph-native reranking using PageRank or centrality algorithms
- Multi-tenancy via Neo4j database-per-tenant (requires Enterprise)

See also: [[ADR-007]], [[ADR-008]], [[ADR-018]]

ADR-010-qdrant

ADR-010 — Qdrant as Vector Database

STATUS

Accepted. Implemented in `platform-search/src/main/java/com/cognitera/platform/search/application/qdrant/QdrantVectorSearchProvider.java`.

CONTEXT

Vector search requires a database optimized for high-dimensional similarity queries. General-purpose databases (PostgreSQL) can support vectors via extensions (pgvector) but are not optimized for vector-first workloads.

DECISION

Use **Qdrant** as the dedicated vector database. Qdrant is purpose-built for vector similarity search with:

- Native cosine similarity scoring
- Payload filtering (metadata alongside vectors)
- Collection management with configurable vector dimensions
- REST + gRPC APIs

The `QdrantVectorSearchProvider` communicates via REST API (`/collections/{name}/points/search`). Each vector point carries a payload with `chunkId`, `documentId`, `title`, `documentType`, `category`, `tags`, and `source`.

Qdrant is **optional**: when `platform.search.qdrant.host` is not configured, `NoOpVectorSearchProvider` provides empty search results and keyword search continues.

Configuration is managed via `QdrantProperties` (`@ConfigurationProperties(prefix = "platform.search.qdrant")`). The default collection is `document_intelligence_chunks` with 768-dimensional vectors (matching `nomic-embed-text` output).

ALTERNATIVES CONSIDERED

- **pgvector (PostgreSQL extension)**: Rejected as primary vector store. While pgvector works for small-to-medium corpora, Qdrant provides better performance at scale, native quantization, and is purpose-built for vector search. pgvector is used for development/testing via H2 compatibility.
- **Weaviate, Pinecone, Milvus**: Rejected. Qdrant was chosen for its Rust performance, simple deployment (single binary), and REST API. The `VectorSearchProvider` SPI makes switching straightforward.
- **In-memory vector search**: Rejected. Does not persist across restarts.

CONSEQUENCES

- **High-performance vector search**: Cosine similarity with payload filtering
- **Collection auto-creation**: `QdrantCollectionManager.ensureCollectionExists()` on startup
- **Batch indexing**: `indexBatch()` for efficient bulk ingestion
- **Graceful degradation**: Platform works without Qdrant (keyword-only search)

TRADE-OFFS

- Additional infrastructure dependency in production
- REST API adds ~5-10ms latency vs gRPC
- No built-in quantization or disk-based indexing in the current configuration

FUTURE EVOLUTION

- gRPC client for lower latency
- Qdrant quantization for memory efficiency with large corpora

- Multi-collection strategy per tenant or document type
- Hybrid search pushdown to Qdrant (if Qdrant adds text search)

See also: [[ADR-008]], [[ADR-011]]

ADR-011-retrieval-orchestration

ADR-011 — Retrieval Orchestration with Intent-Based Strategy Selection

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultRetrievalOrchestrator.java`.

CONTEXT

Different query types benefit from different retrieval strategies. A factual lookup ("What was the revenue in Q2?") benefits from keyword search. An exploratory question ("What capabilities does the platform have?") benefits from hybrid search. An index inspection query should not waste resources on vector search.

DECISION

Implement a **Retrieval Orchestrator** that selects the retrieval strategy based on classified query intent:

```
User Query → Intent Classification → Strategy Selection → Search Execution → Result
```

Strategy mapping (deterministic and explainable):

QueryIntent	SearchMode	Rationale
`INDEX_INSPECTION`, `CORPUS_DISCOVERY`	`KEYWORD`	Exact match queries; vector search unnecessary
`QUESTION_ANSWERING`, `WORKSPACE_ANALYSIS`, `SOURCE_ANALYSIS`	`HYBRID`	Complex queries benefit from keyword + vector fusion
`DOCUMENT_RESEARCH`, `DOCUMENT_LOOKUP`	`SEMANTIC`	Conceptual queries benefit from vector similarity

The orchestrator produces `RetrievalOrchestrationResult` with full **explainability metadata**: intent, strategy, mode, prompt template, result counts, fusion method, reranking status, timing, and a step-by-step `traceLog`.

ALTERNATIVES CONSIDERED

- **Always run all retrievers**: Rejected. Wastes resources on inappropriate strategies (e.g., vector search for exact ID lookups).
- **User-specified strategy**: Rejected. Users should not need to understand retrieval internals.
- **ML-based strategy selection**: Deferred. A trained classifier could outperform keyword-based intent classification, but keyword rules are deterministic, explainable, and sufficient for initial release.

CONSEQUENCES

- **Deterministic**: Same query always produces same strategy
- **Explainable**: Every retrieval decision is recorded in `traceLog`
- **Efficient**: Only appropriate retrievers are executed
- **Prompt-aware**: Retrieves the latest prompt template from `PromptRegistry`

TRADE-OFFS

- Keyword-based intent classification can misclassify queries
- Strategy mapping is static (no runtime adaptation based on result quality)
- Graph retrieval is not automatically selected by the orchestrator (requires explicit `HYBRID_GRAPH` mode)

FUTURE EVOLUTION

- ML-based intent classification for higher accuracy
- Feedback loop: if results are poor, retry with expanded strategy

- Graph strategy auto-selection when enriched entities are detected in the query

See also: [[ADR-005]], [[ADR-008]], [[ADR-012]]

ADR-012-explainability

ADR-012 — Explainability by Default

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/model/RetrievalOrchestrationResult.java` and `platform-ai/src/main/java/com/cognitera/platform/ai/model/InferenceMetadata.java`.

CONTEXT

AI systems must be auditable. When the platform generates an answer, stakeholders need to know: which model generated it, which prompt template was used, which documents were retrieved, which retrieval strategy was selected, and how long each step took. Without explainability, AI output is a black box.

DECISION

Make **explainability a first-class concern** on every inference. Every AI response carries `InferenceMetadata` and every retrieval carries `RetrievalOrchestrationResult` with:

Metadata	Example
`provider`	"ollama"
`model`	"qwen2.5:14b"
`promptTemplateId`	"rag-answer/v1"
`retrievalStrategy`	"HYBRID"
`keywordResultCount`	12
`vectorResultCount`	8
`graphNodeCount`	3
`totalSourceCount`	15
`fusionMethod`	"weighted-linear-fusion"
`rerankingApplied`	true
`rerankingProvider`	"ollama-cross-encoder"
`retrievalStartedAt` / `retrievalCompletedAt`	timestamps
`traceLog`	["Orchestration started", "Intent: QUESTION_ANSWERING", ...]
`evaluationScores`	{grounding: 0.72, faithfulness: 0.85}

The `explain()` method produces a human-readable summary:

```
Intent: QUESTION_ANSWERING | Strategy: HYBRID | Prompt: rag-answer/v1 | Sources: 15 | Fusion: weighted-linear-fusion | Reranking: yes (ollama-cross-encoder) | Duration: 245ms
```

Explainability is **built into the orchestration layer**, not bolted on as an afterthought. Business services don't call explainability APIs — the orchestrator populates metadata automatically.

ALTERNATIVES CONSIDERED

- **Optional explainability**: Rejected. Explainability should never be optional in an enterprise AI platform.
- **Separate explainability service**: Rejected. Would require duplicating orchestration state. Building metadata into the orchestration result ensures consistency.
- **User-facing explainability only**: Rejected. Internal diagnostics are as important as user-facing explanations.

CONSEQUENCES

- **Every inference is auditable**: Full traceability from query to answer
- **Deterministic**: Same query → same strategy → same explainability output
- **Low overhead**: Metadata is collected during normal execution, not as a separate pass
- **Future-proof**: New retrieval sources automatically appear in metadata

TRADE-OFFS

- `traceLog` is append-only string list (not structured log entries)
- `RetrievalOrchestrationResult` uses a builder with 19 setters (verbose but explicit)
- Metadata is not yet exposed through a dedicated API endpoint

FUTURE EVOLUTION

- Structured trace events with typed metadata (not string list)
- Explainability REST API (`GET /api/inferences/{id}/explain`)
- Visualization of retrieval decisions (Sankey diagram of query → strategy → results)
- Differential explainability: "why was document A ranked above document B?"

See also: [[ADR-011]], [[ADR-013]], [[ADR-014]]

ADR-013-evaluation

ADR-013 — Evaluation Engine

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultEvaluationService.java` .

CONTEXT

AI-generated answers must be evaluated for quality. Without evaluation, there is no feedback loop to detect hallucination, poor grounding, or irrelevant answers. Evaluation must run automatically as part of the inference pipeline, not as a separate manual process.

DECISION

Implement an **Evaluation Engine** that automatically evaluates every retrieval + inference cycle:

Metric	Method	Range
`groundingScore`	Context length vs answer length ratio	[0, 1]
`citationCoverage`	Citation markers [1], [2] in answer	[0, 1]
`faithfulness`	Inverse of hallucination indicators	[0, 1]
`answerRelevance`	Term overlap between question and answer	[0, 1]
`contextRelevance`	Term overlap between question and context	[0, 1]
`hallucinationIndicators`	Uncertainty phrase detection	integer ≥ 0
`passed`	Composite quality gate	boolean

The `DefaultEvaluationService` uses heuristic methods (regex patterns, term overlap, length ratios). In production, a dedicated evaluation LLM or framework (deepeval, ragas) would replace these heuristics.

Evaluation is wired into `DefaultRetrievalOrchestrator` — it runs after retrieval and before the result is returned. Every `RetrievalOrchestrationResult` carries evaluation scores in its metadata.

ALTERNATIVES CONSIDERED

- **No evaluation**: Rejected. An AI platform without quality feedback is irresponsible engineering.
- **LLM-as-judge only**: Deferred. Requires a second LLM call, adding latency and cost. Heuristic evaluation provides fast, deterministic feedback. LLM evaluation can be added as a separate evaluation profile.
- **Human evaluation only**: Rejected. Does not scale and cannot run in CI pipelines.

CONSEQUENCES

- **Automatic quality gate**: Every retrieval is scored; low-quality retrievals are flagged
- **Deterministic**: Heuristic methods produce consistent, reproducible scores
- **Lightweight**: Evaluation adds negligible latency (no LLM calls)

TRADE-OFFS

- Heuristic methods are less accurate than LLM-based evaluation
- `hallucinationIndicators` uses regex patterns that miss sophisticated hallucinations
- No benchmark dataset integration for regression testing

FUTURE EVOLUTION

- LLM-as-judge evaluation (separate profile, gated by configuration)
- RAG evaluation framework integration (deepeval, ragas)
- Evaluation benchmark dataset with ground truth annotations
- Automated regression testing: "does this prompt change improve or degrade evaluation scores?"

See also: [[ADR-011]], [[ADR-012]], [[ADR-014]]

ADR-014-workflow-engine

ADR-014 — Workflow Engine

STATUS

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultWorkflowEngine.java`.

CONTEXT

Enterprise AI applications involve multi-step processes: document upload → extraction → enrichment → analysis → review → completion. These processes have defined steps, transitions, and state. Hardcoding step logic in controllers couples workflow to HTTP concerns. A reusable workflow engine separates process definition from execution.

DECISION

Implement a reusable **Workflow Engine** (`WorkflowEngine` interface):

Concept	Implementation
`WorkflowDefinition`	Named workflow with ordered steps and transitions
`WorkflowStep`	Step with id, name, description, handler type (manual/automated/ai)
`WorkflowInstance`	Running instance with current step, status, context

Two pre-registered workflows demonstrate the engine:

1. **`document-intelligence`** : SETUP → INGESTION → ANALYSIS → REVIEW → COMPLETE (5 steps)
2. **`batch-ingestion`** : INGEST → ENRICH → COMPLETE (3 steps)

The engine supports:

- `start(definitionId, context)` — creates a new instance at the initial step
- `advance(instanceId)` — moves to the next step; marks COMPLETED when no further steps

- `previous(instanceId)` — returns to the previous step
- `findInstance(instanceId)` — retrieves instance state
- `listActive()` — finds all active instances

The current implementation is in-memory (suitable for single-node). A database-backed store would be needed for multi-node deployments.

ALTERNATIVES CONSIDERED

- **Hardcoded wizard in controller**: Rejected. The old workspace wizard was a hardcoded switch statement in `WorkspacePageController`. A reusable engine separates process logic from presentation.
- **Camunda/Flowable BPMN engine**: Rejected. Over-engineered for the current use case. The lightweight engine demonstrates the concept without framework lock-in.
- **Spring State Machine**: Rejected. Adds framework dependency for a relatively simple state transition model.

CONSEQUENCES

- **Reusable**: Any module can define and execute workflows
- **Simple API**: 5 methods, intuitive lifecycle
- **Pre-registered workflows**: Demonstrates real usage without configuration files

TRADE-OFFS

- In-memory storage means workflows reset on restart
- No persistence or fault tolerance for long-running workflows
- No parallel step execution or conditional branching
- Step handler types are declarative (manual/automated/ai) but not enforced

FUTURE EVOLUTION

- Database-backed instance storage for production durability
- Conditional transitions based on step outcomes
- Timer-based transitions (escalation after timeout)
- Visual workflow definition (BPMN subset)
- Workflow event publishing for audit integration

See also: [[ADR-001]], [[ADR-020]]

ADR-015-ai-observability

ADR-015 — AI Observability with Micrometer

STATUS

Accepted. Implemented in `platform-observability/src/main/java/com/cognitera/platform/observability/metrics/AiMetrics.java`.

CONTEXT

AI operations are complex, multi-step processes with multiple external dependencies. Without observability, diagnosing failures (is the LLM slow? is Qdrant down? is enrichment producing too few entities?) requires log diving. Production AI systems require metrics.

DECISION

Implement **AI-specific observability** using Micrometer + Prometheus, integrated into a dedicated `platform-observability` module:

Metric	Type	Tags
`ai.inference.duration`	Timer (percentile histogram)	provider, model
`ai.embedding.duration`	Timer	provider, model
`ai.retrieval.duration`	Timer	mode (keyword/semantic/hybrid)

`ai.graph.retrieval.duration`	Timer	—
`ai.enrichment.duration`	Timer	—
`ai.ingestion.duration`	Timer	document_type
`ai.prompt.duration`	Timer	template
`ai.provider.available`	Gauge (0/1)	provider
`ai.embedding.count`	Counter	provider
`ai.enrichment.entities`	Counter	—
`ai.evaluation.*`	Summary	metric name

Metrics are exposed via `/actuator/prometheus` and `/actuator/health`. Health indicators check Ollama, Qdrant, and provider availability.

The `platform-observability` module is reusable across future applications. It depends only on Micrometer and Spring Boot Actuator — no platform-specific dependencies.

ALTERNATIVES CONSIDERED

- **Logging only**: Rejected. Logs are not aggregatable or queryable at scale. Metrics enable dashboards and alerting.
- **OpenTelemetry from day one**: Deferred. Micrometer provides a simpler API that can export to OTLP when OpenTelemetry is adopted.
- **Metrics in each module**: Rejected. Centralizing metrics in `platform-observability` ensures consistent naming, avoids duplication, and enables reuse.

CONSEQUENCES

- **Operational visibility**: Every AI operation produces metrics
- **Reusable module**: Future applications get observability by depending on `platform-observability`
- **Standard format**: Prometheus exposition format enables Grafana dashboards
- **Health endpoints**: Liveness, readiness, and dependency health checks via Actuator

TRADE-OFFS

- Metrics are defined but not yet fully wired into all services (`AiMetrics` is injected in fewer places than ideal)
- No custom Grafana dashboard definitions
- No alerting rules (would be added in deployment configuration, not in the platform)

FUTURE EVOLUTION

- Wire `AiMetrics` into all AI services (`AiService`, `SearchService`, `DefaultEnrichmentService`)
- OpenTelemetry exporter for distributed tracing
- Pre-built Grafana dashboard JSON
- Alert definitions for critical metrics (LLM latency > threshold, provider down)

See also: [[ADR-002]], [[ADR-016]]

ADR-016-graceful-degradation

ADR-016 — Graceful Degradation Over Hard Failures

STATUS

Accepted. Implemented throughout the platform via `@ConditionalOnProperty`, `@ConditionalOnBean`, `ObjectProvider`, and `NoOp*` fallback beans.

CONTEXT

The Enterprise AI Platform depends on multiple external services: Ollama (LLM), Qdrant (vector DB), Neo4j (graph DB), PostgreSQL (relational DB). In production, any of these may be unavailable. The platform must continue functioning with reduced capabilities rather than failing entirely.

DECISION

Design every external dependency as **optional**. The platform uses multiple Spring mechanisms for graceful degradation:

Mechanism	Example	Behavior
`@ConditionalOnProperty`	`OllamaChatProvider` requires `platform.ai.ollama.base-url`	Bean not created if config absent
`@ConditionalOnBean`	`Neo4jGraphSearchAdapter` requires `GraphEnrichmentService`	Bean not created if Neo4j unavailable
`ObjectProvider<T>`	`DefaultDocumentIngestionProcessor` injects `ObjectProvider<EnrichmentHook>`	Null-safe getIfAvailable()
`NoOp*` fallbacks	`NoOpVectorSearchProvider`, `NoOpGraphSearchProvider`	Return empty results, never throw
`try-catch` wrappers	`Neo4jGraphSearchAdapter.search()`	Log warning, return empty list
`isAvailable()` checks	`GraphSearchProvider.isAvailable()`, `ChatCompletionProvider.isAvailable()`	Caller checks before invoking
`ProviderRouter` fallback	4-tier selection skips unavailable providers	Throws only when ALL unavailable

The degradation is **visible**: health indicators report status, metrics track failures, logs record the reason. The platform never silently degrades.

ALTERNATIVES CONSIDERED

- **Mandatory all services**: Rejected. Would prevent development without full infrastructure.
- **Circuit breaker only**: Rejected. Circuit breakers (Resilience4j) are valuable for transient failures but don't address the "service never configured" case. Conditional beans handle both.
- **Feature flags**: Rejected. Adds complexity. Conditional bean activation is simpler and more idiomatic in Spring.

CONSEQUENCES

- **Platform starts with zero infrastructure** (except PostgreSQL for basic operation)
- **157 tests pass without any external services** (H2 in-memory, no Ollama/Qdrant/Neo4j)
- **Clear failure modes**: When a service is down, the reason is logged and surfaced in health endpoints
- **Gradual capability ramp**: Start with keyword search → add Qdrant for vector → add Neo4j for GraphRAG → add Ollama for LLM

TRADE-OFFS

- Some operations silently return empty results (graph search) rather than surfacing errors to users
- No retry logic for transient failures (deferred to Resilience4j integration)
- Health indicators currently read env vars directly rather than Spring config (known drift, documented)

FUTURE EVOLUTION

- Resilience4j integration for retry, circuit breaker, rate limiter
- Health indicators refactored to inject `@ConfigurationProperties`
- Degradation event publishing to audit log
- User-facing capability indicators ("vector search unavailable")

See also: [[ADR-003]], [[ADR-008]], [[ADR-009]], [[ADR-010]], [[ADR-015]]

ADR-017-domain-configuration

ADR-017 — DomainConfiguration for Multi-Domain AI
STATUS

Accepted. Interface defined in `platform-ai/src/main/java/com/cognitera/platform/ai/api/DomainConfiguration.java`. Default implementations embedded in existing services.

CONTEXT

The platform must support multiple application domains (contract intelligence, financial analysis, regulatory compliance, technical documentation) from a single codebase. Domain-specific logic (concept definitions, analysis objectives, finding hierarchies, system instructions) must not be hardcoded in platform services. The platform core must remain domain-independent.

DECISION

Define a **`DomainConfiguration` SPI** that encapsulates domain-specific AI behavior:

Method	Returns	Purpose
`domainId()` / `displayName()`	String	Identity
`concepts()`	`List<ConceptDefinition>`	Keywords + governing references per concept
`objectives()`	`List<ObjectiveDefinition>`	Analysis objectives with keywords
`findingRoleMapping()`	`Map<String, String>`	Reference → finding role mapping
`findingRelationships()`	`List<FindingRelationship>`	Relationships between findings
`centralityWeights()`	`Map<String, Double>`	Centrality weights for references
`peripheralReferences()`	`Set<String>`	References classified as peripheral
`roleKeywords()`	`Map<String, List<String>>`	Source role classification keywords
`systemInstruction()`	`String`	Domain-specific AI system instruction
`answerStructureGuidance()`	`String`	Domain-specific answer structure

Applications provide their `DomainConfiguration` as a Spring bean. The platform's existing services (`DefaultConceptExtractionService`, `DefaultObjectiveAnalysisService`, etc.) currently embed default configurations. These defaults represent the original document intelligence domain but should be migrated to `DomainConfiguration` implementations.

ALTERNATIVES CONSIDERED

- **Hardcode domain logic in each service**: Rejected. Couples the platform to a single domain and prevents reuse.
- **Database-driven configuration**: Deferred. Adds infrastructure dependency. SPI-based configuration is simpler and testable.
- **Remove all domain logic from platform**: Rejected. The platform must demonstrate real AI behavior. DomainConfiguration keeps domain logic while making it swappable.

CONSEQUENCES

- **Extension point exists**: New domains implement `DomainConfiguration` without touching platform code
- **Migration path**: Existing hardcoded domain rules in 8 services can migrate to `DomainConfiguration` implementations incrementally
- **Testability**: Domain configurations can be tested independently

TRADE-OFFS

- Currently a forward-looking SPI — the 8 existing services still embed their configurations rather than consuming `DomainConfiguration`
- No multi-domain routing (which `DomainConfiguration` to use for a given query)
- No domain discovery mechanism

FUTURE EVOLUTION

- Migrate 8 services to consume `DomainConfiguration`
- Multi-domain routing based on query classification
- Domain configuration discovery (`List<DomainConfiguration>` injection for multi-domain setups)
- External domain configuration via YAML/JSON files

See also: [[ADR-003]], [[ADR-007]]

ADR-018-provenance-graph

ADR-018 — Provenance-Aware Knowledge Graph

STATUS

Accepted. Implemented in `platform-neo4j/src/main/java/com/cognitera/platform/neo4j/model/GraphNode.NodeProvenance`.

CONTEXT

Knowledge graphs generated from AI extraction are only trustworthy if every node and edge can be traced back to its source. Without provenance, a graph node claiming "Acme Corporation is an ORGANIZATION" cannot be audited — was this extracted from a financial report or hallucinated by an LLM?

DECISION

Every graph node and relationship carries **`NodeProvenance`**:

Field	Purpose
`sourceDocumentId`	Which document was the source
`chunkId`	Which chunk within the document
`chunkOffset`	Position within the chunk
`extractionConfidence`	How confident was the extraction (0.0–1.0)
`extractionTimestamp`	When was it extracted
`extractionModel`	Which model performed the extraction
`promptVersion`	Which prompt template version was used
`provider`	Which AI provider performed the extraction

Provenance is captured during enrichment (`DefaultEnrichmentService`), bridged through `EnrichmentHook` , and persisted to Neo4j by `GraphEnrichmentService` . The data flows: `EnrichmentContext` → `EnrichmentResult` → `GraphNode` (with `NodeProvenance`) → Neo4j.

ALTERNATIVES CONSIDERED

- **No provenance**: Rejected. An unauditable knowledge graph is useless for enterprise applications.
- **Separate provenance table in PostgreSQL**: Rejected. Graph data should carry its own provenance. A separate store creates consistency challenges.
- **Blockchain-based provenance**: Rejected. Over-engineered. Immutable audit log in PostgreSQL + provenance in Neo4j provides sufficient traceability.

CONSEQUENCES

- **Full traceability**: Every graph element can be traced to its source document, chunk, model, prompt, and provider
- **Confidence-aware**: Extraction confidence enables filtering low-confidence extractions
- **Reproducibility**: Re-extraction with different models/prompts can be compared by timestamp and version

TRADE-OFFS

- Provenance adds storage overhead to every node and relationship
- `extractionConfidence` is self-reported by the extraction method (not independently verified)

- Provenance is not yet leveraged for retrieval scoring (e.g., boost high-confidence extractions)

FUTURE EVOLUTION

- Confidence-weighted retrieval: boost documents with high-confidence extractions
- Provenance visualization in the UI
- Automated re-extraction when prompt version changes
- Provenance chain: "entity A was extracted from chunk B of document C by model D using prompt E at time F"

See also: [[ADR-007]], [[ADR-008]], [[ADR-009]]

ADR-019-testing-strategy

ADR-019 — Multi-Layer Testing Strategy

STATUS

Accepted. Implemented across `platform-ai/src/test/` and `platform-api/src/test/`. Documented in `TESTING.md`.

CONTEXT

An Enterprise AI Platform requires confidence that every architectural subsystem behaves correctly. Tests must validate behavior, not implementation details. A reader should understand the platform architecture by reading the tests.

DECISION

Adopt a **5-layer testing strategy**:

Layer	Location	Execution	Purpose
Unit	`platform-ai/src/test/.../unit/`	`mvn test`	Validate components in isolation
Integration	`platform-api/src/test/.../ai/`	`mvn verify`	Validate subsystems with Spring context
Architecture	`platform-api/src/test/.../architecture/`	`mvn verify`	Validate end-to-end behavior
Contract	`platform-ai/src/test/.../contract/`	`mvn verify`	Validate SPI stability
Playwright (UI)	`e2e-tests/playwright/`	`mvn verify -Pui-tests`	Validate browser behavior

Key principles:

- **Behavior over implementation**: Tests verify what the platform does, not how it does it
- **Executable documentation**: Test names describe behavior (`"routes openai:gpt-4o to openai provider"`)
- **Realistic scenarios**: Architecture tests use a `test-corpus/` with real document types and expected behaviors
- **Contract tests**: Every `ChatCompletionProvider` implementation must satisfy the abstract `ChatCompletionProviderContract`
- **No mocks for platform internals**: Unit tests mock external providers; integration tests use real Spring context with H2

157 tests validate: Prompt Registry (12), Model Capability Registry (12), Provider Router (14), Provider Contracts (5), Retrieval Orchestrator (3), Evaluation Engine (8), Workflow Engine (10), Semantic Enrichment (14), Graceful Degradation (9), GraphRAG (4), AI Pipeline (10), Browser UI (12), Platform (91).

ALTERNATIVES CONSIDERED

- **Coverage-driven testing**: Rejected. Coverage percentage does not measure architectural confidence. 157 behavioral tests provide more value than 500 getter tests.
- **BDD framework (Cucumber)**: Rejected. Adds framework complexity. JUnit 5 `@DisplayName` + `@Nested` provides sufficient behavior description.

- **No UI tests**: Rejected. Browser tests validate user-visible behavior that unit tests cannot.

CONSEQUENCES

- **Architecture as tests**: Test structure mirrors platform architecture
- **Confidence, not coverage**: Every major subsystem has executable proof of correctness
- **CI-ready**: `mvn clean verify` runs all tests except Playwright (separate profile)
- **Contract protection**: New providers must satisfy SPI contracts

TRADE-OFFS

- Playwright tests require a running application (separate Maven profile)
- `test-corpus/` is small (3 documents) — sufficient for architectural validation, not exhaustive
- No performance regression tests yet

FUTURE EVOLUTION

- Performance smoke test suite (`mvn verify -Pperformance`)
- Extended test corpus with multilingual documents
- Snapshot tests for prompt rendering output
- Automated evaluation regression: "did this prompt change degrade faithfulness?"

See also: [[ADR-020]], TESTING.md

ADR-020-platform-philosophy

ADR-020 — Enterprise AI Platform Design Philosophy

STATUS

Accepted. Embodied in every architectural decision across the platform.

CONTEXT

The platform is not a chatbot. It is not a RAG demo. It is a reference implementation of an Enterprise AI Platform designed to be reusable across multiple AI-powered applications. Every architectural decision flows from this philosophy.

DECISION

The platform is governed by these design principles:

####### 1. AI is Infrastructure AI is not a feature. It is infrastructure, like databases or message queues. Business logic depends on abstract interfaces (`ChatCompletionProvider` , `EmbeddingProvider`), not concrete implementations (`OllamaChatProvider` , `OpenAiChatProvider`). Adding a new AI provider requires zero changes to business logic.

####### 2. Modular Monolith Over Microservices Module boundaries are enforced at compile time. Clear dependency direction prevents cycles. The architecture supports future extraction into microservices but does not prematurely distribute.

####### 3. Graceful Degradation is Mandatory Every external dependency is optional. The platform starts with zero infrastructure and gains capabilities as services become available. Keyword search works without Qdrant. Retrieval works without Neo4j. Inference works without Ollama.

####### 4. Explainability by Default Every AI operation produces auditable metadata: which model, which prompt, which strategy, which documents, how long it took. Explainability is not a feature toggle — it is built into the orchestration layer.

####### 5. Documents Are the Knowledge Source Knowledge is extracted from documents automatically during ingestion. There is no manual knowledge base, no CRUD interface for entities. The enrichment engine extracts entities, concepts, and relationships. The knowledge graph is auto-generated.

####### 6. Domain Independence The platform core contains no domain-specific logic. Domain customization happens through `DomainConfiguration` implementations. The platform can serve contract intelligence, financial analysis, compliance, or technical documentation from the same codebase.

####### 7. Testing as Architecture Documentation Tests validate behavior, not implementation. Test structure mirrors platform architecture. A Principal Engineer should understand the platform by reading the test suite.

####### 8. Production Readiness Every subsystem considers: how does it fail? How is it observed? How is it configured? Conditional beans, health indicators, metrics, structured logging, and configuration properties are not afterthoughts — they are first-class concerns.

ALTERNATIVES CONSIDERED

The alternative philosophies considered and rejected:

- **"Move fast and break things"**: Incompatible with enterprise AI. Every decision is deliberate, documented, and testable.
- **"Maximize features, minimize architecture"**: Would produce a prototype, not a platform. The architecture is the product.
- **"AI is magic"**: Treating AI as opaque magic leads to unmaintainable systems. Every AI operation is instrumented, evaluated, and auditable.

CONSEQUENCES

- **Reusable platform**: Future applications (contract intelligence, financial analysis, compliance) can be built on the same foundation
- **Clear extension points**: `DomainConfiguration`, `ChatCompletionProvider`, `GraphSearchProvider` define where the platform grows
- **Professional engineering**: The codebase demonstrates architecture, testing, observability, and resilience — not just AI integration
- **9 modules, 157 tests, 0 failures**: Every architectural subsystem participates in real execution paths

TRADE-OFFS

- Architectural rigor means more interfaces and abstractions than a prototype would have
- The platform is not optimized for the shortest path to a demo — it is optimized for long-term maintainability
- Some SPIs (`DomainConfiguration`) are forward-looking and not yet consumed by all eligible services

FUTURE EVOLUTION

This document should remain stable. New ADRs should reference these principles. If a proposed change conflicts with a principle, it should either be rejected or this ADR should be amended with explicit rationale.

See also: [[ADR-001]], [[ADR-003]], [[ADR-012]], [[ADR-016]], [[ADR-017]], [[ADR-019]]
